UNIT D — Turing Machines and Decidability

Study Notes (Book Chapter)

Module: TCR117V — Theoretical Computer Science V (Advanced Diploma, FoICT, TUT) **Outcome covered:** D.1 — Describing Turing machines and decidability (what computers can do); D.2 — Analysing the Halting Problem **Assessment criteria:** D.1.1 – D.1.6, D.2.1 – D.2.3 **Semester:** 2, 2026

How to use these notes

This chapter is written as a textbook chapter. Each of the nine assessment criteria (D.1.1 to D.1.6, D.2.1 to D.2.3) forms its own major section, so you can study criterion by criterion:

Section	Assessment criterion	Topic
1	D.1.1	The Turing machine — anatomy, components, and why it is the model of every computer
2	D.1.2	Variants (multi-tape, non-deterministic) and why extra features do not change what is computable
3	D.1.3	The Universal Turing Machine and the stored-program (von Neumann) architecture
4	D.1.4	Turing-recognisable versus Turing-decidable languages

Section	Assessment criterion	Topic
		(recogniser vs decider)
5	D.1.5	Determining whether a real-world problem is decidable
6	D.1.6	Decidability in real software — guarantees, heuristics and testing
7	D.2.1	The Halting Problem — statement and why it matters to developers
8	D.2.2	Why the Halting Problem is undecidable — diagonalisation and reduction, step by step
9	D.2.3	The practical consequences — why perfect tools cannot exist
10	—	Chapter summary and criteria checklist
11	—	Glossary of key terms
12	—	Self-assessment questions (answers at the end)
13	—	Exam-style question bank (model answers at the end)

Section	Assessment criterion	Topic
14	—	Sources and further reading

Every section follows the same pattern: a **real-world scenario** first, then a **simple analogy** you can tell a child, then the **concepts** built up from first principles, then **worked examples**, then a **conclusion-first summary**, and finally a **“Where you will use this”** box that links the theory to the working world. Key structures are drawn as figures (the Turing machine and its worked program, the equivalence of variants, the Universal Turing Machine, the recognisable-versus-decidable worlds, the decidability check, the Halting Problem, diagonalisation, reduction, and the practical consequences for software). The mathematics in this unit — configurations, simulation, encodings of programs as data, countability and the diagonal argument — is explained step by step from the very beginning: nothing is assumed, and nothing is skipped. Work through the **self-check questions** as you go — they are designed to show you exactly what you would be asked in a test or exam.

Before you begin — the big picture

In Unit A you met the five features that make a machine a computer, and the claim that one abstract object — the **Turing machine** — is the pure form of every computer. In Unit B you met the models of computation — the Turing machine, the lambda calculus, the RAM model — and Church’s thesis: every reasonable model of computation captures exactly the same set of computable functions. In Unit C you learned to measure how *efficiently* a computation runs. This unit asks the question that sits underneath all of them: **what can a computer do at all?**

The answer has two halves, and they are both surprising.

The first half is reassuring: *almost everything you have ever programmed is doable.* One single, deliberately primitive machine — a tape, a head, a few states and a table of rules — can compute anything any computer can compute. Fancier machines (extra tapes, guessing, quantum-looking parallelism) add speed and convenience but never new abilities. And one fixed machine, the **Universal Turing Machine**, can run *any* program, which is the theoretical identity card of every laptop, phone and cloud server you will ever touch.

The second half is unsettling: *some things no computer can do — and we can prove it.* The most famous is the **Halting Problem**: given any program and any input, determine whether the program will ever stop running. No program can answer that question for all programs, ever — not today, not with faster hardware, not with more cleverness. That single result, proved by Alan Turing in 1936, draws the boundary of the entire computing profession: it is why no tool can perfectly detect infinite loops, why no compiler can guarantee your program is correct, and why software engineering is a discipline of *evidence* (testing, analysis, review) rather than certainty.

By the end of this chapter you should be able to look at any problem in your work and answer the question this whole unit exists for: **can an algorithm solve this problem with a guaranteed answer for every input — or is it one of the problems for which no such algorithm can exist?** And if it is one of those, you should know exactly what professionals do instead.

---

1. The Turing machine — the abstract model of every computer (D.1.1)

> **Assessment criterion D.1.1:** A Turing machine is described and its components are explained using the analogy of a real computer program, recognising it as the abstract model of every computer.

1.1 Scenario: the shop that lost its manual

You are the developer at a small logistics company. The boss walks in holding a faded, dog-eared ring-binder. It is the 1980s manual for the company's ancient mainframe — the only copy, and the only description of how it decides which parcels share a delivery route. The machine still works; nobody alive knows how. The boss asks you to write a new system that does exactly what the old one does. You have no source code, no documentation beyond the manual, and no way to run experiments that don't affect live deliveries.

What you actually need is a way to *describe a computation precisely* — to say, in a language with no ambiguities, exactly what "the machine does" means. And that is exactly the problem Alan Turing was solving in 1936, years before any electronic computer existed: **what is the most precise possible description of a computation?** His answer became the most important single idea in computer science — the **Turing machine** — and it is the same answer you would use today to specify what your delivery-router must do before you write a line of code.

1.2 Simple analogy: the checkerboard robot

Tell this to a six-year-old:

Imagine a robot that walks along a very long sidewalk made of square tiles. Each tile has a symbol written on it — a 0, a 1, or a blank. The robot has a tiny brain with a few "moods": it can be in the happy mood, the thinking mood, or the done mood. It follows ONE rule at a time, written on a card the grown-up holds: "IF you are in the happy mood AND you are standing on a tile with a 1, THEN rub out the 1, write a 0, take one step to the right, and switch to the thinking mood."

The robot does not think. It does not plan. It just reads the card, checks its mood and the tile under its feet, and does exactly what the card says — then reads the card again. When

it reaches the “done” mood, it stops. Change the cards, and the same robot does a completely different job: one set of cards makes it count, another set makes it sort letters, another makes it check whether a word is spelled the same forwards and backwards.

The grown-ups’ trick is this: the robot itself never changes. The *cards* are the program. The sidewalk is the memory. And because the cards are just pieces of paper, another robot — or the same robot — can be given any cards at all. That is the whole secret of computers: **a fixed machine that follows whatever instructions you give it.**

In grown-up terms: the sidewalk tiles are the *tape*, the robot’s mood is the *state*, the cards are the *transition function* (the program), and the one-tile step is the *head movement*. Alan Turing’s genius was to notice that this toy — tape, head, states, rules — is powerful enough to do *any* computation that any computer can do, and simple enough to reason about perfectly. Every modern computer is, at bottom, this robot.

1.3 From first principles: what exactly is a computation?

Before the machine, we need the thing it computes. From Unit A you know the picture: a computation takes **input**, applies rules, and produces **output**. But “rules” is vague. Turing’s contribution was to make “rules” so precise that a machine could follow them without any intelligence at all.

Start with the raw materials:

- **Symbols.** Everything a computation touches is a symbol: letters, digits, spaces, punctuation — and two special ones: a *blank* (the empty tile) and the *start marker*. The collection of symbols the machine may write is its **alphabet**. A real program’s “alphabet” is enormous (all Unicode characters); a Turing machine gets away with a tiny alphabet — often just {0, 1, blank} — because everything else can be encoded (Section 3 explains how).
- **Strings.** A **string** is a finite sequence of symbols — the word `HELLO`, the number `1011`, a file’s contents. The input to a computation is always a string; the output is always a string.
- **A problem as a language.** Here is the step that makes theory and practice meet. From Unit A, a *language* is a set of strings. And every yes/no problem can be seen as a language: the language of a problem is the set of all strings for which the answer is “yes”. Example: the problem “is this string a palindrome?” has as its language the set of all palindromes — `ABA`, `RACECAR`, `101101` — and the machine’s job is to say “yes” exactly for members of that set. Every program you have ever written is, in this sense, a device that separates the “yes” strings from the “no” strings.

1.4 The machine itself: four components

A Turing machine (TM) is a mathematical object with exactly four parts:

1. **The tape.** An infinite strip divided into cells, each holding exactly one symbol from the alphabet (or the blank). The tape is the memory. “Infinite” means *unbounded*: no

matter how much space the computation needs, there is always more tape. Real computers approximate this with RAM and disk.

2. **The head.** A read/write device positioned over one cell. In one step it (a) reads the symbol under it, (b) writes a symbol there (possibly the same one), and (c) moves one cell left or right. The head is the CPU's point of contact with memory — it can only see one location at a time, exactly like a single CPU core executing one instruction at a time.

3. **The states.** A finite set of internal conditions the machine can be in — including one **start state**, and one or two **halting states** (accept and reject). The state is the machine's entire "memory of what happened so far". Note the word *finite*: there may be only, say, 10 states — yet the machine's behaviour can still be unboundedly complex, because the tape can grow and because states are revisited.

4. **The transition function.** The program. It is a finite table of rules of the form:

   > *If the machine is in state q and the head reads symbol a, then: write symbol b, move the head (L or R), and enter state q′.*

   Given the current state and the current symbol, the table names the new symbol, the move, and the next state. Nothing else. That one rule format — "read, decide, write, move, change state" — is the entire vocabulary of computing.

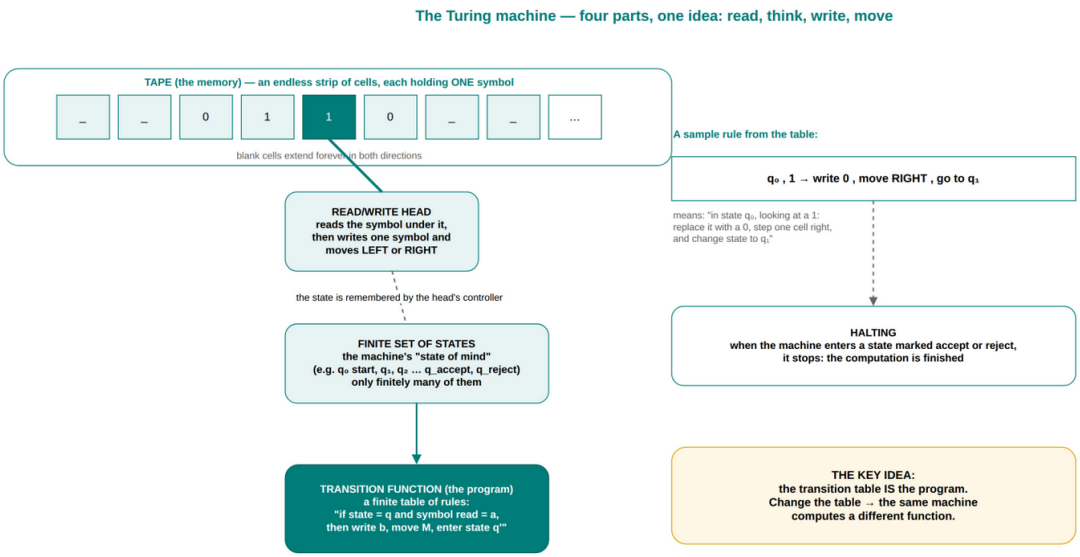

Figure 1 — The Turing machine: an infinite tape of cells, a read/write head, a finite set of states, and the transition function — the program.

The machine **computes** as follows: place the input string on the tape, position the head over the leftmost input symbol, set the state to the start state, and then apply the transition function repeatedly until the machine enters a halting state. If it halts in an **accept** state, the input is accepted (the answer is "yes"); if it halts in a **reject** state, it is rejected ("no"). If it never halts — the third possibility — the machine *runs forever*: it has

not answered at all. That third possibility is not a bug in the definition; it is the single most important fact in this unit (Sections 4, 7 and 8).

1.5 The TM as a program: every part maps to your code

The assessment criterion asks you to explain the components *using the analogy of a real computer program*. The mapping is exact, not approximate:

Turing machine component	Real computer equivalent	Why the analogy holds
Tape	RAM, disk, memory-mapped files	Both are linear storage of symbols; the TM's infinite tape is just RAM that never runs out
Head	CPU register / program counter	One location visible at a time; the machine reads and writes through this single point
States	Program counters, function-call stacks, flags	The finite "where am I and what do I know so far" memory of the running program
Transition function	The machine code / the program itself	A finite list of rules: "if in state X with value Y, do Z" is precisely `if`-statements and assignments
Read a symbol, write, move	Load, compute, store, branch	The fetch–decode–execute cycle from Unit A, in miniature
Accept / reject states	`return true;` / `return false;`	The program's final answer
Running forever	An infinite loop	The program never answers

Here is the crucial point to internalise: the transition function is not *like* a program — **it is a program.** A program in Python that loops over a list is doing, in effect, what the TM does: read a value, decide, maybe write, move on, repeat. The TM is simply the smallest, most rigid version of that idea — so small that mathematicians can prove things about it, and so faithful that everything proved about it is true of your Python code too.

1.6 A worked Turing machine: the parity checker

Theory is nothing without a machine that actually runs. Here is a complete, working Turing machine — the smallest meaningful program you will ever meet. The problem: **given a string of 0s and 1s, accept it if it contains an even number of 1s, reject it otherwise.** (Think: a parity check — the same check a network card runs on every packet it receives.)

The machine uses two states: q_0 (start; “even number of 1s seen so far”) and q_1 (“odd number of 1s seen so far”). Alphabet: {0, 1, blank}. The transition function has six rules:

State	Symbol read	Write	Move	Next state	Meaning
q_0	0	0	R	q_0	A 0 does not change the parity
q_0	1	1	R	q_1	A 1 flips the parity to odd
q_1	0	0	R	q_1	A 0 does not change the parity
q_1	1	1	R	q_0	A 1 flips the parity back to even
q_0	blank	blank	—	**accept**	Reached the end with even parity
q_1	blank	blank	—	**reject**	Reached the end with odd parity

Run it on the input `101` (two 1s — even — so it should accept):

- **Start:** state q_0, head on the first `1`. Rule 2 fires: write 1, move right, enter q_1. (One 1 seen: odd.)
- **Step 2:** state q_1, head on `0`. Rule 3 fires: write 0, move right, stay q_1. (0s never change the count.)
- **Step 3:** state q_1, head on the second `1`. Rule 4 fires: write 1, move right, enter q_0. (Two 1s seen: even again.)
- **Step 4:** state q_0, head on the first blank. Rule 5 fires: **accept**.

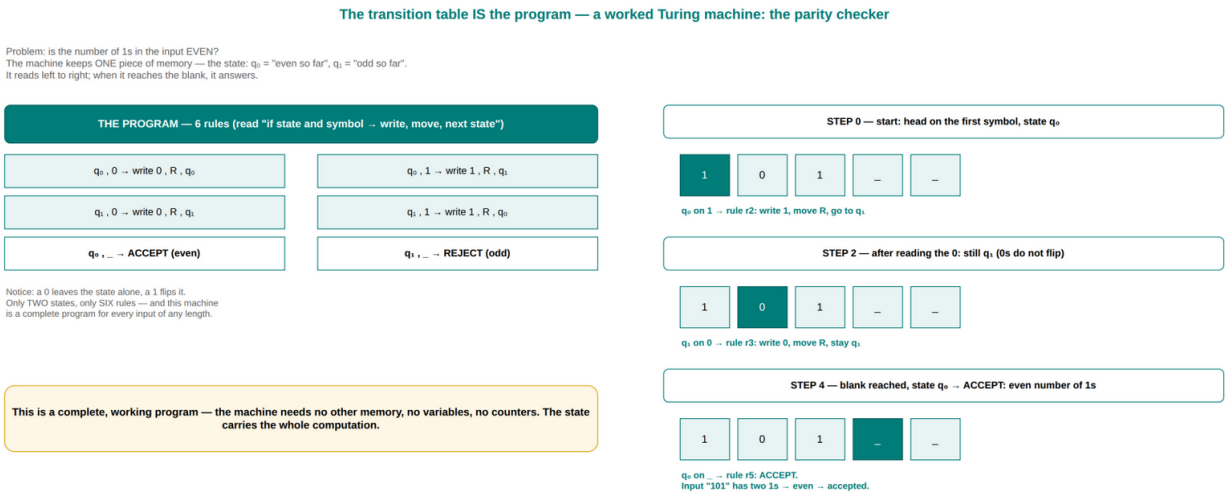

Figure 2 — A worked Turing machine: the parity checker's six rules and the step-by-step trace of input "101".

Notice what did *not* happen: the machine never counted to two. It has no counter, no variable, no memory besides the state. The parity was carried entirely in *which state it was in* — "even" and "odd" are all the information the job requires. That is the TM style of programming: no cleverness, no shortcuts, just read, decide, write, move, change state. And yet this same style, scaled up, implements every algorithm ever written.

1.7 Why this machine is the model of every computer

Two questions now arise, and the rest of the unit answers them:

1. *Is this machine too weak?* A tape and six rules cannot run my banking app… or can it? The answer is the **Church–Turing thesis** (Unit B): every computation that any reasonable model can do, a Turing machine can do. Your banking app's computation is a finite sequence of symbol-manipulation steps, and a TM can perform that sequence — slowly, awkwardly, but completely. No model of computation that has ever been proposed (lambda calculus, RAM machine, register machines, quantum computing under the standard model) computes more than the Turing machine. This is not a theorem in the usual sense — it is a thesis, a scientific claim confirmed by a century of failed counterexamples — but it is accepted by every computer scientist.
2. *Is this machine too strong?* Could a TM compute things a real computer cannot? No: every real computer is *finite*, while the tape is infinite, so the TM is the idealisation "a computer whose memory never runs out". Everything a real machine computes, a TM computes; a TM may additionally compute things that would run out of memory on any real machine, but for *feasible* computations the two coincide. That is why results about the TM are results about every real computer — including the impossibility results of Sections 7–9.

> **Conclusion-first (D.1.1):** A **Turing machine** is the mathematically pure form of a stored-program computer: an **unbounded tape** (memory) divided into cells, a **head** (the CPU's single point of contact) that reads and writes one symbol at a time, a **finite set of states** (the machine's memory of what happened so far, including accept and reject), and a **transition function** — a finite table of "read, decide, write, move, change state" rules that **is the program**. The machine runs by applying the rules to the input until it accepts, rejects, or runs forever. Because the transition table is just data, the same machine can hold any program; and by the Church–Turing thesis, any computation any computer can do is a computation some Turing machine can do. This one toy — tape, head, states, rules — is the abstract model of every computer that exists.

1.8 Where you will use this (D.1.1)

- **Specifying software precisely:** before writing an algorithm, describing it as a clear read–decide–write–repeat loop catches ambiguities that prose hides. The TM

discipline — “what exactly is my state, what exactly is my data, what exactly does each rule do” — is the discipline of writing formal specifications.
- **Understanding any program deeply:** when you debug, you are tracing a state machine — variables are the state, each statement is a transition. Programmers who see programs this way read code faster and write fewer bugs.
- **Protocols and parsers:** network protocols, regex engines and tokenisers are often literally implemented as machines with states and transitions (Unit F makes this concrete).
- **Proving what software can and cannot do:** everything from Sections 4–9 stands on this machine; recognising “I am dealing with a Turing machine-shaped problem” tells you whether a guaranteed solution can exist at all.

1.9 Self-check (D.1.1)

1. List the four components of a Turing machine and the real-computer part each one models.
2. In one sentence, what is the “program” of a Turing machine?
3. Write the transition-function rules (in table form) for a TM that accepts exactly the strings that start with a 1, over the alphabet {0, 1}.
4. Trace your machine from Question 3 on the input `101` and on the input `011`, saying at every step which rule fires and why the machine accepts or rejects.
5. Explain why “the machine may run forever” is part of the definition, not a bug.
6. (Challenge) Why is the tape infinite, when real memory is finite? What would break in the definition if the tape were finite?

---

2. Fancier machines, same power: why variants change nothing (D.1.2)

> **Assessment criterion D.1.2:** The equivalence of the standard Turing machine and its variants (e.g. multi-tape, non-deterministic) is explained, along with why adding features does not change what is computable.

2.1 Scenario: “if only we had a faster bus”

Your team maintains the API for a bank. Every night a batch job runs: for each of 40 000 customers, it must check their transactions against 2 000 fraud rules. The job takes 3 hours. A junior developer proposes a redesign: instead of one slow process, split the work across *ten machines* — one per 4 000 customers — and run them in parallel. The manager asks you: “will this make the system able to compute things it couldn’t before?”

The answer is no — and it is important to say why clearly. Parallelism makes the *same* computations finish **faster**; it does not create any computation that was previously impossible. Everything the ten-machine cluster can do, the original single machine could

do too — just slower. Speed is a *complexity* question (Unit C). Ability is a *computability* question (this unit). The two get confused constantly, and this section exists to separate them, using the Turing machine's own "fancier versions" as the laboratory example.

2.2 Simple analogy: the kitchen and the two cooks

Tell this to a six-year-old:

Mama is cooking supper for a big family. She has one pot, one stove plate and one chopping board, and she works alone: chop, stir, taste, wait. It takes an hour. Then Grandmother arrives and offers to help. Now there are two pots, two plates, and two chopping boards — but Grandmother is not a better cook, and she cannot make any *new* dish. She can only make the same dishes, faster. Chicken tonight? Still chicken. The only difference is *how long* it takes and *how much room* is needed in the kitchen.

Now imagine Grandmother is not just a helper but a *guesser*: whenever the recipe says "if you are not sure whether the onions are cooked, taste them" — instead of tasting, she just *guesses* "yes!" and keeps going. That is still cooking. It may finish faster (if the guesses are lucky), but she cannot invent a dish the recipe never allowed. Same kitchen, same recipes, always: **more helpers and lucky guessing change the time, never the menu.**

In grown-up terms: the kitchen is the Turing machine; extra pots are *extra tapes*; Grandmother is a *second head*; guessing is *non-determinism*. The menu — the set of computations that can be performed — never changes. Only the *speed* and the *convenience* change. That is exactly the theorem this section proves: every variant of the Turing machine computes exactly the same set of functions as the plain single-tape machine.

2.3 From first principles: what "computable" means, and what "the same" means

Two ideas need to be made exact before the variants make sense.

First: what does "the machine can compute a problem" mean? A Turing machine M *computes the function* f if, whenever you place the string x on its tape, M halts with f(x) written on the tape. It *decides the language* L if it always halts — accepting exactly the strings in L, rejecting the rest. And a problem is **computable** (or decidable) if *some* Turing machine computes/decides it. One machine that works for all inputs — that is the definition of a solution.

Second: what does it mean for two machines to be "the same"? Two machines (or two *models* of machine) are **equivalent** if they compute the same set of functions — no more and no less. If everything the fancy machine computes, the plain machine also computes, and vice versa, then in the computability sense they are the *same machine*, no matter how different they look. This is exactly the equivalence idea from Unit B (the Church–Turing thesis family): we judge models by their abilities, not their appearances.

There is one more primitive idea we need: **simulation**. Machine A *simulates* machine B if A can follow B's behaviour step by step — reading what B reads, writing what B writes, accepting when B accepts. Simulation is how we prove equivalence: to show the plain machine can do what a fancy machine does, we build a plain machine that simulates the fancy one. And simulation is possible only because of the deepest fact from Section 3: **a machine's behaviour is a string of symbols, so a machine can read the description of another machine and copy it.** You simulate a Playstation on a laptop; a TM simulates another TM.

2.4 Variant 1: the multi-tape Turing machine

A **multi-tape Turing machine** has k tapes (k a fixed number like 2, 3 or 100) and k independent heads — one per tape. In one step it reads all k symbols, writes on all k tapes, moves all k heads, and changes state. Why would anyone build such a thing? For the same reason a human uses scratch paper: one tape holds the input, a second tape holds notes ("which letter am I looking for"), a third holds the answer. Sorting, copying, comparing — all become dramatically easier to *design* when you can keep your working notes separate from your input.

The theorem: **a multi-tape TM computes exactly what a single-tape TM computes.** Why? Because a single-tape machine can simulate the multi-tape one. The trick — worth understanding once, it explains a thousand simulations — is to store all k tapes on the one tape, stacked in *tracks*, and to mark each head's current position with a special symbol. One tape cell now holds a little column: the first track's symbol, a head marker (or not), the second track's symbol, and so on. To simulate one step of the multi-tape machine, the single-tape machine sweeps from leftmost head position to rightmost head position, reading all k symbols, and sweeps back, writing them all. More work per step — but the work is finite, and it all fits on one tape.

The cost is worth counting, because it is the bridge to Unit C: if the multi-tape machine takes t steps, the simulation takes at most about t^2 steps — a **quadratic slowdown**. Quadratic is a *complexity* difference, not a *computability* difference: the single-tape machine still halts exactly when the multi-tape machine halts, so it accepts exactly the same languages. Slower, yes. Weaker, no.

2.5 Variant 2: the non-deterministic Turing machine

A **non-deterministic Turing machine (NTM)** differs from the standard machine in one explosive detail: at some steps, the transition function offers *several* possible rules for the same (state, symbol), and the machine may choose any of them. It is as if the machine could branch into many parallel copies of itself, each exploring a different rule — and the computation *accepts* if any branch reaches an accept state.

Why would anyone define such a thing? Because search problems are natural to express this way. "Is there a route through 200 cities that costs less than R5 000?" — a deterministic machine must tediously try routes one after another; a non-deterministic machine *guesses* a route and checks it. Guessing-then-checking is the definition of the

class **NP** from Unit C — the N in NP literally stands for “non-deterministic”. So the NTM is not a curiosity; it is the machine that defines the most famous open question in computer science.

The theorem again: **an NTM computes exactly what a deterministic TM computes.** The proof is a simulation too — a three-tape construction. The simulator performs a **breadth-first search** over all branches of the NTM’s computation: on tape 2 it keeps a queue of “addresses” of active branches, and on tape 3 it simulates one branch from the address. If any branch accepts, the simulator accepts. This takes exponentially more time in the worst case (up to 2^t steps to simulate t non-deterministic steps) — but it halts exactly when some branch would have accepted, so it decides exactly the same languages. Exponential slowdown again: a *complexity* difference, never a *computability* one.

2.6 The full equivalence picture

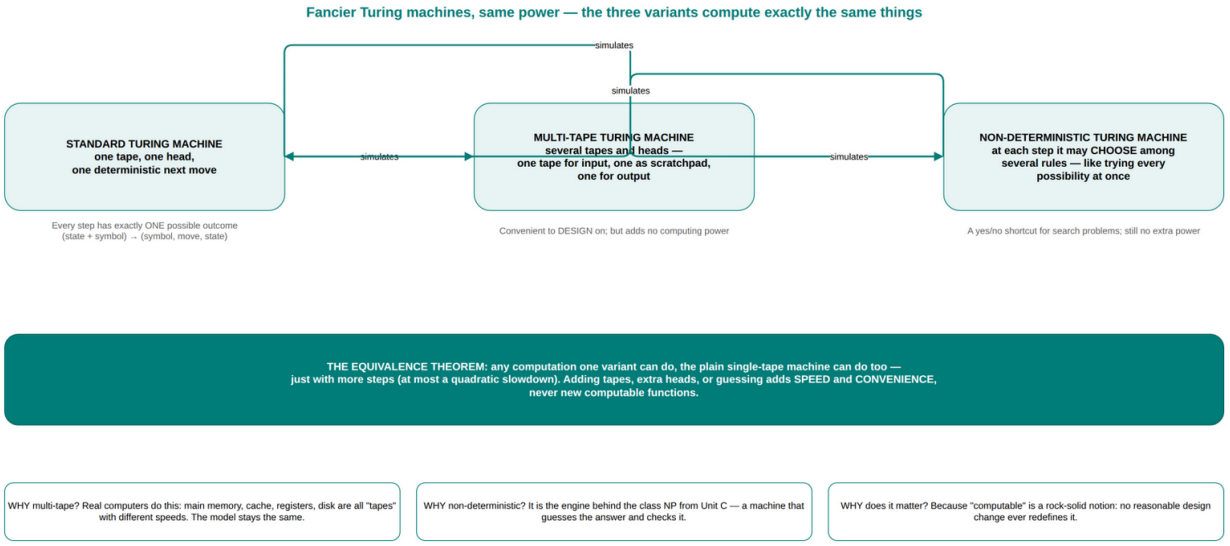

Figure 3 — The three variants and their simulations: every variant computes exactly the same functions as the plain single-tape machine.

The pattern should now be visible, and it is the pattern the whole subject relies on:

Variant	What it adds	What it costs	Does it add computable functions?
Multi-tape	Scratch tapes, parallel heads	Quadratic slowdown	No
Non-deterministic	Branching / guessing	Exponential slowdown	No
Random access	Jump to any tape cell by address	Quadratic slowdown	No
	A grid instead of a line	Quadratic slowdown	No

Variant	What it adds	What it costs	Does it add computable functions?
Two-dimensional tape			
Fixed finite alphabet	Any alphabet you like	Constant factor	No

Every “improvement” that has ever been proposed to the Turing machine reduces to the same story: it changes the *time* and the *convenience*, and it changes *neither* the set of computable functions *nor* the set of decidable languages. This is the mathematical content of the phrase in the criterion: **adding features does not change what is computable.**

Why is this so robust? Because every variant is itself a machine — finite description, mechanical rules, unbounded memory — and by the Church–Turing thesis every such machine is simulable by a plain TM. The simulation theorems turn out to be surprisingly small and mechanical once the encoding trick of Section 3 is available. The fancy machine’s rules are just data; the plain machine reads the data and obeys it.

2.7 Why this matters: what “computable” means once and for all

This robustness is the reason the field can use the word “computable” as an absolute, timeless notion:

- When you prove in Unit E that some problem is *not* computable, the proof is about the plain TM — and because all variants are equivalent, the result applies to *every* model: your laptop, a quantum computer, a machine from the year 2100, any future hardware. “Not computable” is a property of the problem, not of the current technology.
- When a vendor promises that a new chip architecture, a GPU, or a quantum processor can “compute things no computer could before”, the honest translation is *“computes some things much faster”* — which is valuable — but never *“computes things that were impossible”*.
- The P versus NP question from Unit C is only well-defined because of this equivalence: NP is defined via non-deterministic machines, P via deterministic ones, and the question “are they the same class?” has a stable meaning precisely because both classes sit on top of the same underlying notion of computation.

Conclusion-first (D.1.2): A **variant** of the Turing machine is any machine that keeps the same idea — finite control, unbounded memory, mechanical rules — but adds conveniences such as extra tapes, parallel heads, random access or non-deterministic guessing. Every such variant is **equivalent** to the plain single-tape machine: each can **simulate** the others, accepting exactly the same languages and computing exactly the same functions. The simulations cost extra time (quadratic for most variants, exponential for non-determinism), so variants change **efficiency** and **ease of design** — never **power**. This is why "computable" is a rock-solid, technology-independent notion: no reasonable design change ever redefines it.

2.8 Where you will use this (D.1.2)

- **Architecture decisions:** "add more cores / GPUs / specialised chips" — evaluating such proposals correctly means separating *speed* (they change it) from *capability* (they cannot change it).
- **Algorithm design:** choosing to express a search as "guess and check" (the NTM view) clarifies the structure of an NP problem before you design the deterministic algorithm or heuristic that implements it.
- **Reading research papers:** the sentence "without loss of generality, consider a multi-tape TM" appears everywhere; knowing it is a convenience, not a restriction, lets you read the results correctly.
- **Understanding why no future machine escapes the limits of this unit:** equivalence is the reason Sections 7–9's impossibility results are permanent.

2.9 Self-check (D.1.2)

1. State precisely what it means for two models of computation to be equivalent.
2. Explain, in your own words, how a single-tape TM simulates a multi-tape TM. Where does the quadratic slowdown come from?
3. What is non-determinism, and why is it connected to the class NP from Unit C?
4. Why does non-determinism cause an exponential slowdown when simulated deterministically?
5. A colleague claims: "With 100 tapes, we can finally compute the halting function." Respond in two sentences, using the equivalence theorem.
6. (Challenge) Why is "same computable functions" the right notion of equivalence rather than "same running time"? Give an example of two machines that are equivalent but have wildly different running times.

---

3. The Universal Turing Machine — one machine that runs any program (D.1.3)

> **Assessment criterion D.1.3:** The Universal Turing Machine is described, and an explanation is made of its relationship to the stored-program (von Neumann) architecture of modern computers — a single machine that runs any program.

3.1 Scenario: one phone, every app

Hold up your phone. It contains one processor — one fixed piece of silicon — yet this afternoon it has been a messaging device, a maps device, a banking device, a camera, a games console and a translator. Nothing about the silicon changed between those uses; only the **stored instructions** changed. When you install an app, you are not installing a new computer; you are writing new data into memory that a fixed machine will obey. That fact — so familiar it has become invisible — is precisely the most profound theoretical result in this unit: **a single fixed machine can execute any program.** Turing proved it in 1936, a decade before the first stored-program computer existed; John von Neumann read Turing's paper and turned the idea into the architecture every computer still uses. This section connects the abstract proof to the silicon in your pocket.

3.2 Simple analogy: the book-reading machine

Tell this to a six-year-old:

There is a machine — big and grey, with a slot in the front. It does only one thing: it *reads books out loud*. Not one book — any book. You slide a cookbook in: it reads the cookbook, out loud. You slide a storybook in: it reads that one, too. You slide the manual for a spaceship in: it reads that as well, and it has never seen a spaceship.

The machine does not contain a thousand books. It contains one skill — "follow the words on the page" — and the books themselves are *just paper*. The paper is not part of the machine; it is something the machine reads. Give it a new book and the same machine performs a new trick, without a single screw being turned.

The cleverest bit: one of the books can be a book about *another book-reading machine*. The machine reads it and then does exactly what that other machine would have done with its own books. It can even read a book about itself. The grey box is one thing; the books make it anything.

In grown-up terms: the grey box is the **Universal Turing Machine** — a fixed machine whose only special skill is *interpreting descriptions of other machines*. The books are **programs**: strings of symbols that describe computations. Because a program is just data (a string on the tape), the universal machine can read any program and then do exactly what that program describes. "One machine runs any program" is not marketing

language; it is the content of a theorem, and it is the theoretical skeleton of the von Neumann architecture: fixed CPU, programs in memory, memory holds data.

3.3 From first principles: programs are strings — the encoding idea

The universal machine seems to be pulling itself up by its own bootstraps: a machine that runs "any program" would need to contain every program. It cannot — and it does not need to, because of one simple observation:

> **A program is a string of symbols.** Any program — a Turing machine, a Python script, an Excel macro — can be written down as a finite sequence of symbols. A Python file is text; a compiled binary is bytes; a Turing machine's transition table is a list of rules, and a list of rules is a string.

Because programs are strings, they can be *placed on a tape* — they are ordinary data. Turing formalised this with an **encoding**: fix a rule that writes any TM's transition table as a string of 0s and 1s, and write ⟨M⟩ for "the encoding of machine M". Then ⟨M⟩ is a string like any other — it can be copied, stored, compared, and handed to other machines as input. This is exactly what a compiler does (a program that consumes programs as data) and what an operating system does when it loads an `.exe` (instructions read into memory as bytes).

Two consequences follow immediately, and the entire rest of this unit stands on them:

1. **Machines can be inputs to machines.** If ⟨M⟩ is a string, then a machine can *receive* ⟨M⟩ on its tape and process it. Programs analysing programs — compilers, linters, antivirus scanners, the tools of Unit E — are all just machines whose input happens to be a description of another machine.
2. **The list of all machines is countable.** Because every machine has a finite encoding, the machines can be put in a list: M_1, M_2, M_3, … — the list of all finite strings that happen to describe valid machines (throw away the ones that don't parse). This "listability" is what makes the diagonalisation argument of Section 8 possible. It is also the first time in this module that we count *programs* — the count turns out to be the smallest infinity (Section 8.5 does the counting properly).

3.4 The Universal Turing Machine, defined

Turing's construction — the crown jewel of the 1936 paper — is now only a few steps away:

> **Universal Turing Machine (UTM):** a fixed Turing machine U that takes as input a tape containing ⟨M⟩ (the encoding of any machine M) followed by x (any input string), and computes exactly what M would compute on x: it halts with the same

> output, accepts exactly when M accepts, rejects exactly when M rejects, and runs forever exactly when M runs forever.

How does U do this without being M? The same way an interpreter works: U is a machine with a few special states and a few dozen rules that *read the description ⟨M⟩ and then obey it step by step*:

1. U examines the current symbol on its tape.
2. U looks through ⟨M⟩'s transition table for the rule matching (M's current state, current symbol) — U keeps M's current state written on a section of its own tape, like a variable.
3. U performs the rule's action: writes the new symbol, moves the head, updates M's state (or halts if the rule says accept/reject).
4. U repeats.

Nothing magical: U is just a very careful reader of very small print. The description ⟨M⟩ is the data; U's own fixed rules are the interpreter; together they behave exactly like M. This is the same relationship as a Python interpreter (fixed program) running a Python script (data) — the interpreter does not contain the script's behaviour; it *performs* it.

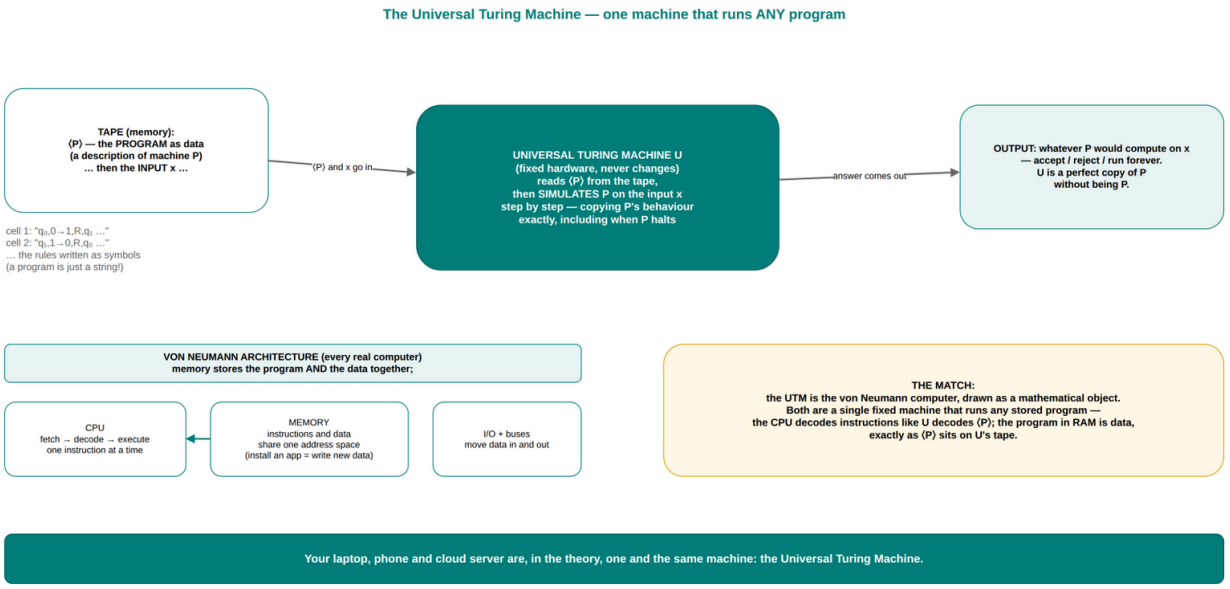

Figure 4 — The Universal Turing Machine: a fixed machine reads the encoding of any program from its tape and simulates it — the theoretical form of the von Neumann architecture.

One subtle point is worth naming, because it is the seed of everything in Sections 7 and 8: the UTM can feed a machine to *itself*. ⟨U⟩ is a string, so U can read a tape containing ⟨U⟩ and simulate itself. And ⟨M⟩ can contain the string ⟨M⟩: a machine can be given a description of itself as its own input. This **self-reference** looks like a parlour trick and is in fact the hinge on which the whole theory of undecidability turns.

3.5 The UTM is the stored-program computer: the von Neumann architecture

Now look at a modern computer with the UTM in mind:

von Neumann component (Unit A)	Role	Universal-machine counterpart
Memory — one store for instructions and data	Holds the program and its data together	The tape: ⟨M⟩ (instructions) and x (data) on one tape
Control unit	Fetches the next instruction, decodes it, dispatches it	U's rules: locate the next rule of ⟨M⟩, interpret it, act on it
ALU	Performs arithmetic and comparisons	The few "do the rule" states of U (arithmetic itself is simulated)
Registers	A few fast working locations	The tape section where U keeps M's current state and head position
I/O	Moves data between machine and world	Reading the initial tape, writing the final tape
Program loading	Loads a stored program into memory	Reading ⟨M⟩ off the tape at the start of the simulation

The correspondence is not decorative; it is historical. John von Neumann's June 1945 *First Draft of a Report on the EDVAC* described the stored-program computer — and von Neumann explicitly credited Turing's 1936 universal-machine paper, which he had read. Turing had shown that a machine could store the description of a computation in its own memory and then execute it; von Neumann built exactly that with electronics. Every machine since — EDSAC, the iPhone, the server farm — is, in the theory, a Universal Turing Machine wearing a different hat.

The two claims in the criterion, then, are one claim:

> **A stored-program computer is a physical UTM.** "One machine runs any program" (the UTM) and "instructions and data share one memory" (von Neumann) are the same idea stated abstractly and concretely. The CPU decodes instructions

> exactly as U decodes ⟨M⟩; the program in RAM is data exactly as ⟨M⟩ sits on the tape; installing an app is writing new data into memory exactly as handing U a new ⟨M⟩.

3.6 Three consequences that will not let go of you

1. **Emulation is a theorem, not a feature.** Every machine can run every other machine's programs (subject to speed). Emulators, virtual machines, Docker containers, Java's bytecode interpreter — all are practical universal machines. When QEMU runs a Windows program on a Linux PC, it is performing, in miniature, exactly U's trick.
2. **Programs can be analysed as data — which is why the Halting Problem is even possible.** Because a program is a string, the question "does program P halt on input x?" is a question *about strings*, and it can be posed to a machine. The tragedy of Section 8 is not that the question cannot be asked; it is that it *can* be asked perfectly and yet cannot be answered.
3. **There are more problems than programs.** The UTM shows every program is a finite string; Section 8.5 shows the problems outnumber the strings. That mismatch — too many problems, too few programs — is the root of every undecidability result in Units D and E.

Conclusion-first (D.1.3): The **Universal Turing Machine** is a single fixed Turing machine that reads the **encoding ⟨M⟩** of any other machine off its tape and **simulates it step by step**, so that one machine behaves like every machine. It works because **programs are strings** — a program can be stored, copied, and handed to another machine as ordinary data. The UTM is the theoretical form of the **stored-program (von Neumann) architecture**: a fixed control unit that fetches and decodes instructions from a memory holding both instructions and data. Every computer ever built is a physical UTM; emulation, compilers and program-analysis tools are its everyday expressions; and the same "program is data" fact makes self-reference possible — the mechanism behind the undecidability results that follow.

3.7 Where you will use this (D.1.3)

- **Operating systems and toolchains:** loading a program into memory and running it — the OS's central act — is UTM behaviour; so are linkers, loaders and just-in-time compilers.
- **Virtualisation and containers:** VMs and Docker run arbitrary guest programs on a fixed host kernel; the guarantee that this is possible *in principle* is the UTM.
- **Program-analysis tooling:** linters, code review bots and security scanners all consume programs as data — the encoding idea in daily use.
- **Interviews and design discussions:** "explain how one computer can run every program" is the UTM; the answer your interviewer wants is exactly this section.

3.8 Self-check (D.1.3)

1. What is an encoding ⟨M⟩, and why must it be a finite string?
2. Describe, in four steps, how the UTM simulates an arbitrary machine M.
3. Give three real-world technologies that are “practical universal machines”.
4. Match each von Neumann component to its UTM counterpart in a table.
5. Why is self-reference (a machine receiving its own encoding as input) possible — and why will it matter?
6. (Challenge) Explain why the UTM proves that the set of all programs is countable. What would break if programs were *not* listable?

---

4. Recognised, decided, or impossible: three worlds of languages (D.1.4)

> **Assessment criterion D.1.4:** The concepts of Turing-recognisable and Turing-decidable languages are distinguished, and an explanation is made of the practical difference: a decider always gives an answer, while a recogniser may run forever.

4.1 Scenario: the support ticket that never closes

A support engineer is investigating a service that has been “processing” one customer’s migration for three days. The log shows the job started, and nothing else. The team has a rule: a migration is allowed to run for four hours; beyond that, it is killed and re-queued. Nobody knows whether the job would ever have finished — it might have completed at hour 100, or it might have looped forever.

Now think about what the team actually knows, and what it does not. It knows the job has not *answered* yet. It does not know whether an answer will ever come. And the crucial point: **no amount of waiting can distinguish “slow” from “forever”.** The team’s four-hour timeout is a *practical* line, not a *logical* one. This scenario is not a sad corner case — it is the fundamental situation of the recogniser in this section: a machine that says “yes” when the answer is yes, but that may *never say anything at all* when the answer is no. The distinction between a machine that always answers and a machine that may hang is the distinction between **decidable** and merely **recognisable** — and it is one of the most practically important distinctions in this module.

4.2 Simple analogy: the treasure hunter and the mailbox

Tell this to a six-year-old:

Two friends are looking for treasure. Naledi searches every spot on the map in order, and when she has checked them all she comes back and says — always, no matter what — either “I found it!” or “I checked everywhere, it is not there.” She always comes back with an answer. Thabo searches differently: he follows sparkly paths wherever they lead, and he only comes back when he finds treasure. If the treasure exists, he will find it and come back saying “found it!” But if the treasure does *not* exist, Thabo never comes back at all — he just keeps following sparkly paths forever. He never says “it is not there.” He says *nothing.*

If the treasure exists, both friends eventually report “found it”. The difference appears when the treasure does not exist: Naledi says “no”, Thabo says nothing — forever. Naledi is the **decider**; Thabo is the **recogniser**.

In grown-up terms: a language is *decided* by a machine that always halts — accepting the members and rejecting the non-members, every time, no exceptions. A language is *recognised* by a machine that halts-and-accepts the members but may loop forever on the non-members. Every decided language is recognised (a decider is also a recogniser — it just has better manners), but not every recognised language is decided. The treasure-hunter question — “does Thabo’s silence mean the treasure is absent?” — is exactly the question you cannot answer in practice: *waiting can never distinguish “no” from “not yet”.*

4.3 From first principles: languages, and what a machine does with them

From Unit A: a **language** is a set of strings (a set of yes-answers to some question). From Section 1: a Turing machine processes a string and either accepts, rejects, or runs forever. Combine them and three notions fall out:

> **Turing-recognisable (recursively enumerable, RE):** a language L is *Turing-recognisable* if there is a TM M that accepts every string in L. On strings not in L, M may reject *or may run forever* — the definition does not care, and that is the point.

> **Turing-decidable (recursive):** a language L is *Turing-decidable* if there is a TM M that accepts every string in L **and rejects every string not in L** — that is, M **halts on every input**. Such a machine is called a **decider**.

Read those two definitions side by side and you will see the entire difference is one word: the recogniser may run forever on non-members; the decider may not. That single word — *may* — is the boundary between two worlds, and the rest of this unit is about why the boundary is real.

Why is the recogniser definition even worth having? Because the world is full of machines that are recognisers but not deciders, and the most important of them is the machine that *simulates other programs*. To recognise the language “⟨M⟩ x — M halts on x” (the Halting language), the recogniser is trivial: run U, simulate M on x, and if M halts, accept. If M

never halts, the simulator never halts — and by Section 8, *no* decider for this language exists. So the recogniser is the best that can be done for that language, and it is the reason the word “recognisable” exists: it names what we can still achieve when deciding is impossible.

4.4 The practical difference: an answer, or an eternally spinning hourglass

The practical difference between the two worlds is exactly the scenario’s lesson, and it deserves to be stated as crisply as possible:

	Decidable (a decider)	**Recognisable but not decidable (a recogniser)**
On a “yes” input	Halts and accepts	Halts and accepts
On a “no” input	Halts and rejects — **always, quickly or slowly, but definitely**	May run forever — **you never get a “no”**
What you can rely on	A guaranteed answer for every input	A guaranteed “yes” when the answer is yes; otherwise: silence
Real-world feel	A well-specified function call: it returns	A job with no timeout: it either completes or hangs

The working consequence: with a decider you can build systems that depend on the answer — “reject the payment”, “block the request”, “flag the row”. With a mere recogniser you can only wait — which is why real systems wrap recognisers in timeouts and watchdogs (Section 9), accepting that “no answer within 5 seconds” is not the same as “no”.

One more subtlety that examiners love: **the recogniser’s behaviour on non-members is allowed to be “reject *or* run forever” — and the machine can be a mixture.** A recogniser may reject some non-members outright and loop on others. The definition does not promise *which*. So you cannot “upgrade” a recogniser by waiting a bit longer or adding a clock — the loop may be unbounded. Waiting is not a decision procedure; this is precisely why the timeout is a practical necessity rather than a proof.

4.5 The three worlds in one picture

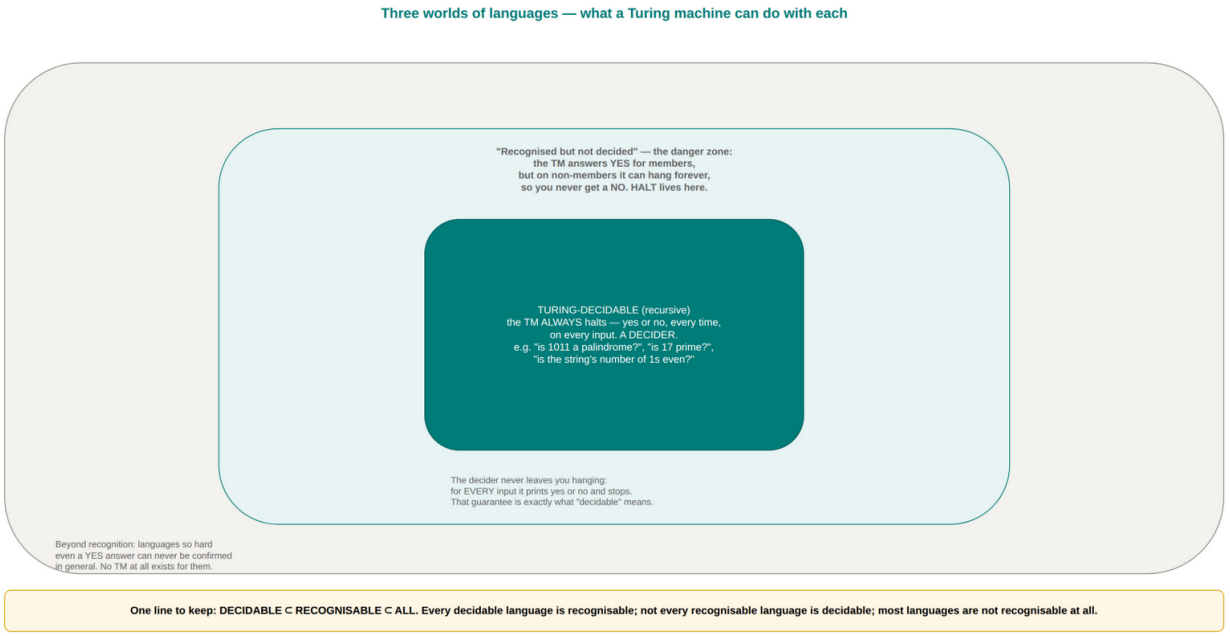

Figure 5 — Three worlds of languages: decidable ⊂ recognisable ⊂ all languages — and where the famous examples live.

The diagram shows three nested worlds. The names of the languages in each world are worth memorising with a reason attached:

World 1 — decidable (the inner teal box). Languages with deciders: “is this string a palindrome?” (the parity-checker-style TM always halts), “is this number prime?” (trial division terminates), “is this string a well-formed email address?” (a regex engine terminates), “does this graph have a path from A to B?” (a search terminates). What they have in common: a *total* algorithm — a recipe that finishes on every input. These are the problems you can ship as functions with guaranteed answers.

World 2 — recognisable but not decidable (the middle band). Languages with recognisers but no deciders — the most important being the **Halting language** HALT = {⟨M⟩ x : M halts on x}. The simulator recognises it (simulate, accept on halt) but — Section 8 — nothing decides it. This band is the danger zone of Section 4.1: the recogniser answers “yes” beautifully and says *nothing* otherwise.

World 3 — not even recognisable (the outer grey region). Languages for which even the “yes” answer cannot be guaranteed — no TM at all exists. The classic example is the **complement of HALT**: {⟨M⟩ x : M does *not* halt on x}. If a recogniser for *this* existed, we could combine it with the HALT recogniser to decide HALT (a machine running both would always get one “yes” answer) — and Section 8 says that is impossible. So the complement of HALT sits outside the recognisable world entirely: not just undecidable, but *unrecognisable*.

The nesting is strict: **decidable ⊊ recognisable ⊊ all languages.** The first inclusion is a theorem in one line (a decider is a recogniser); the strictness of both inclusions comes

from Section 8; and “most languages are in the outer world” comes from the counting argument of Section 8.5. The picture is the map of everything this unit (and Unit E) is about.

4.6 Why the distinction is the point of the whole unit

The decidable/recognisable boundary is not a piece of bookkeeping; it is the formal shape of a professional truth:

- A decidable problem is one you can **promise** to solve. Contracts, APIs and SLAs are built from decidable problems: “this endpoint returns within the timeout or raises an error” is a decidable-style guarantee.
- A merely recognisable problem is one you can **work towards but never complete**. “Has this program ever halted?” is the prototype; every infinite-loop hunt, every “is the migration done?” ticket, is an encounter with the recogniser’s silence.
- The boundary also organises what to do next in your career: when a requirement turns out to be recognisable-but-not-decidable, the professional response is not to wait harder — it is to redesign the requirement (restrict the input class), or to accept a timeout-based answer (Section 9). Knowing *which* world you are in tells you which response is even possible.

> **Conclusion-first (D.1.4):** A language is **Turing-recognisable** if some TM accepts exactly its members (and may reject or loop on everything else); it is **Turing-decidable** if some TM **halts on every input**, accepting members and rejecting non-members. A machine with the second property is a **decider** — it always gives an answer; a machine with only the first is a **recogniser** — it may run forever on non-members, so a missing answer can never be distinguished from a “no”. Every decidable language is recognisable (a decider is a recogniser), but the reverse fails: HALT is recognisable yet not decidable, and its complement is not even recognisable. The practical difference is a guarantee: decidable = guaranteed answer for every input; recognisable-only = guaranteed “yes”, possible eternal silence otherwise — which is why real systems time out recognisers.

4.7 Where you will use this (D.1.4)

- **Service design:** deciding whether a job endpoint can *guarantee* completion (“decider”) or must run with timeouts (“recogniser”) determines the whole reliability architecture — kill switches, dead-letter queues, idempotency.
- **Report and query design:** “will this query always return?” is a decidability question; query planners and query limits are the practical answer when it is not.
- **Analysing vendor promises:** “our tool detects every possible infinite loop” claims decidability; the theory says it cannot be true — which tells you exactly what to audit.

- **Exam readiness:** the decider/recogniser distinction is the most tested concept in this unit; the two definitions above, verbatim, are the answer to the question that appears every year.

4.8 Self-check (D.1.4)

1. Give the definitions of Turing-decidable and Turing-recognisable, and state the one-word difference between them.
2. Why is every decidable language recognisable, but not vice versa?
3. Explain, in terms of Section 4.1, why waiting can never distinguish “slow” from “forever”.
4. Classify each language as decidable, recognisable-but-not-decidable, or not recognisable: (a) palindromes over {0,1}; (b) HALT; (c) the complement of HALT; (d) strings with an even number of 1s.
5. Describe the recogniser for HALT (the simulator) and explain precisely where it fails to be a decider.
6. (Challenge) Suppose someone offers you a “halting oracle” that answers HALT instantly. Which languages become decidable in a world with such an oracle, and which remain unrecognisable?

---

5. Decidable or not? Determining it for real-world problems (D.1.5)

> **Assessment criterion D.1.5:** A determination is made of whether a given real-world problem is decidable by identifying an algorithm that always terminates, or by explaining why no such algorithm can exist.

5.1 Scenario: the requirements meeting that needed a theory

Your team is scoping three features for a fintech product. The product owner lists them:

1. **Feature A:** given two account balances and a transfer amount, decide whether the transfer should be allowed (sufficient funds, no limit breach).
2. **Feature B:** given the codebase of any third-party library and its documentation, decide whether the library will ever crash on any input a customer could give it.
3. **Feature C:** given a log file, decide whether it came from a crashed run of the system.

Feature A is a routine calculation. Feature B is… different. The team debates adding a “crash detector” to CI that *guarantees* no library will ever crash. Feature C is a pattern-matching task. A senior engineer says: “A is easy, C is easy, B is impossible — not ‘hard’, *impossible*.” This section gives you the discipline to say that sentence with confidence —

and to say exactly *why* — by teaching the standard four-step determination procedure that every software professional can run in a requirements meeting.

5.2 Simple analogy: the four questions of the toy factory

Tell this to a six-year-old:

A toy factory makes machines from cardboard boxes and string. The boss wants to know, for each machine idea, “will this machine always do its job?” The factory’s engineer asks four questions in order:

1. **What exactly is the job?** “Always do its job” is too vague — is it “put the ball in the hole” or “put the ball in the hole without touching the sides”? A question you cannot state precisely cannot be answered at all.
2. **Is there a recipe?** Can we write down steps for the job — a list that, followed exactly, does it? If nobody can even write the recipe, there is nothing to check.
3. **Does the recipe always finish?** Even if the recipe exists, does it ever get stuck — marching in a loop forever, waiting for something that never comes?
4. **Is the recipe allowed to use the machine’s own plans?** Here is the trap: some jobs are stated *about machines themselves* — “will this machine ever stop?” — and those jobs are exactly the ones where the recipe can never be finished, no matter how clever the engineer is.

The engineer can answer “yes, guaranteed” for the ball-and-hole machine. But for the job “check whether any machine built in this factory will ever jam” — the answer is: no recipe can ever exist. Not “nobody has found one” — *there cannot be one*. The factory’s most important tool is knowing which jobs are like that.

In grown-up terms: the four questions are (1) *is the problem a well-posed yes/no question about finite input?* (2) *does a candidate algorithm exist?* (3) *does it always terminate — worst case included?* and (4) *does the problem ask something about programs themselves?* (the class that contains the undecidable ones). Question 4 is where the Halting Problem and its relatives live. The determination procedure is these four questions, run in order, and it is the entire content of criterion D.1.5.

5.3 From first principles: what “an algorithm that always terminates” really means

Three notions need to be sharpened before the procedure, because each is a place where half-understanding causes wrong determinations.

A problem, precisely. A *problem* is a pair of things: a set of **instances** (the inputs) and a yes/no question about each instance. “Is this number prime?” — instances: natural numbers; question: prime? “Does this route cost less than R500?” — instances: routes; question: under budget? The instance is always *finite* data — a number, a string, a file, a

graph. This is the discipline of Question 1: if you cannot say what an instance is and what makes an answer “yes”, you are not looking at a well-posed problem.

An algorithm, precisely. An **algorithm** is a finite, unambiguous recipe that takes an instance and produces the answer. The word *finite* matters: the recipe itself is a fixed, finite text — it does not grow with the input. The word *unambiguous* matters: every step has exactly one meaning (this is exactly the transition-function discipline of Section 1).

Termination, precisely. An algorithm **terminates** on an instance if it runs for finitely many steps and stops with an answer. An algorithm *always* terminates if it terminates on *every* instance — including the biggest, nastiest, most adversarial ones. This is the strongest, and the only honest, guarantee: “it terminated on the test cases” is not “it always terminates”. A problem is **decidable** exactly when *some* algorithm exists that always terminates with the right answer. Note what this definition does *not* require: speed. An algorithm that takes a billion years per instance still proves decidability (that is the divide between this unit and Unit C — decidable ≠ feasible, and Section 9 leans on the difference).

5.4 The determination procedure — four questions, in order

Question 1 — Is it a yes/no problem about finite input? If the requirement cannot be stated as “given X, answer yes or no” with X finite, then it is not a well-posed problem yet — rephrase it before going further. (“Find the best route” becomes “is there a route under budget R?”; “make the app safe” becomes “does every payment flow end in an authorisation decision?”). If it cannot be rephrased — e.g. it asks about *all possible future inputs* at once — it is outside the reach of decision entirely.

Question 2 — Does a candidate algorithm exist? Write one down, or identify one from the toolbox: search, sort, compare, count, verify, parse, simulate. This is ordinary engineering — the same “is there a recipe?” question you ask before every implementation. The discipline is to write the recipe *before* claiming it answers the problem: a vague “we could probably check it” is not an algorithm.

Question 3 — Does it always terminate? Examine the candidate for the three classic non-termination traps: - **Unbounded loops:** a loop whose exit condition never becomes true for some inputs (“while not found” over an unbounded search). - **Unbounded recursion:** recursion whose depth has no bound tied to the input’s finite size. - **Simulation of arbitrary programs:** the special trap — if the algorithm’s job is to *run another program and observe something about it*, then its own termination depends on the simulated program’s behaviour, which the algorithm does not control. This trap, and only this trap, is where undecidability lives (Question 4).

If the candidate passes Question 3, the problem is **decidable** — you have exhibited a total algorithm, and that is the determination.

Question 4 — Can it be about programs themselves? (the undecidability probe) If the problem asks a question *about programs or their behaviour* — does it halt, does it ever

print X, does it contain a bug, does it match another program's behaviour, does it satisfy its specification — then the danger flag goes up: problems in this family are often undecidable, and the determination "no algorithm can exist" must be *proven* by **reduction** from a known undecidable problem (the recipe for doing that is Section 8.6). The absence of a proof is not a determination: "we haven't found one" ≠ "none exists". The honest report for an undecided problem is: *decidability unknown* — which is itself a legitimate professional finding (the honesty box in Figure 6).

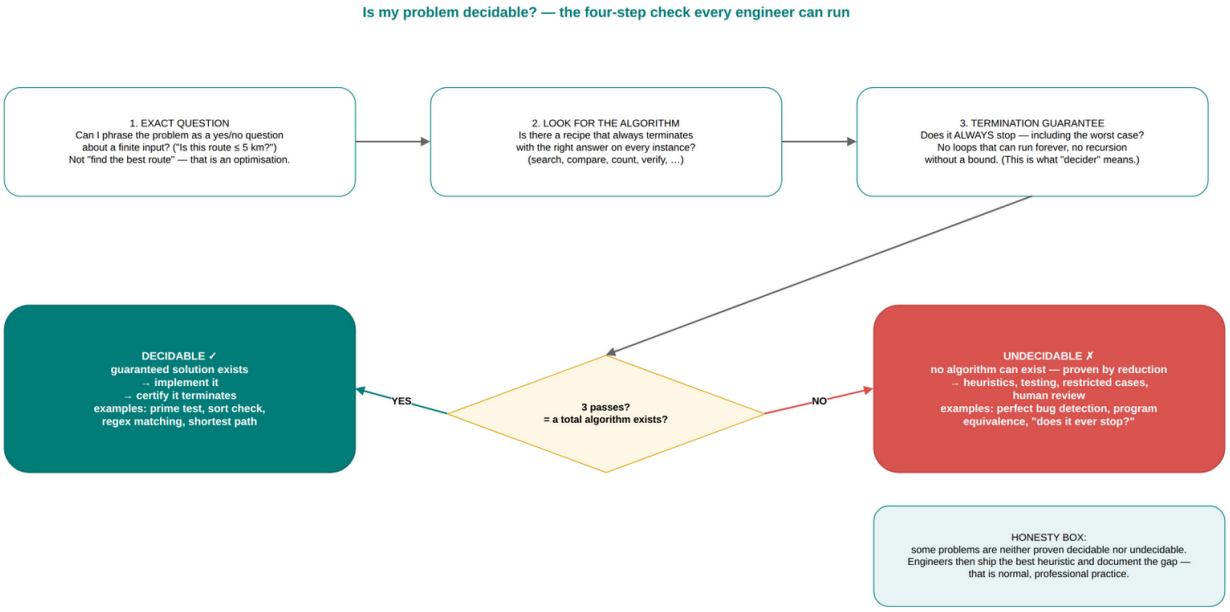

Figure 6 — The four-step determination: exact question → find the algorithm → verify termination → probe for program-behaviour questions.

5.5 Worked determinations — the four cases

Worked example 1 — the transfer check (Feature A). Instance: ($balance_1$, $balance_2$, amount). Question: is $balance_1 - amount \geq 0$ and within the limit? Algorithm: subtract and compare — three arithmetic steps, no loops. Termination: obvious — always. **Determination: decidable.** (Guaranteed solution exists: implement it.)

Worked example 2 — regex matching. Instance: (pattern, text). Question: does the text match the pattern? Algorithm: run the regex engine (an NFA simulation — Unit F). Termination: the NFA simulation always halts (it processes the text left to right, finitely many positions). **Determination: decidable.**

Worked example 3 — route feasibility (TSP decision). Instance: (cities, distances, budget R). Question: is there a tour visiting all cities costing ≤ R? Algorithm: try every ordering of the cities and check. Termination: yes — n! orderings, each checked in finite time, then stop. **Determination: decidable** — and the exercise in Unit C is precisely about *how expensive* the deciding algorithm is. Decidable, but possibly infeasible; that is the permanent difference between D.1.5's "decidable" and C.1.6's "efficiently solvable".

Worked example 4 — the crash detector (Feature B). Instance: (program P, allowed inputs). Question: will P ever crash on an allowed input? Probe Question 4: this asks something *about P's behaviour*. The determination is **undecidable** — by reduction (Section 8.6 shows the standard reduction: if a crash-detector existed, we could build a HALT-decider by wrapping programs so that they crash exactly when they would halt). Note the correct professional response to Feature B: do not promise a guaranteed crash-detector; deliver static analysis with documented false alarms, fuzzing, and a "no crashes found on these 10^6 inputs" report — the honest tools of Section 9.

5.6 A decision table you can reuse

Problem	Instance	Candidate algorithm	Terminates always?	Determination
Is this number prime?	Integer n	Trial division up to √n	Yes	**Decidable**
Is this string a palindrome?	String s	Compare from both ends	Yes	**Decidable**
Does this text match this regex?	(regex, text)	NFA simulation	Yes	**Decidable**
Is there a tour under budget R?	(cities, distances, R)	Try all orderings	Yes (n! steps)	**Decidable** (infeasible for large n)
Is this program free of bugs?	(P, spec)	— (would need P's behaviour)	—	**Undecidable** (by reduction)
Does this program ever halt on x?	(P, x)	Simulate P on x	No — loops if P loops	**Undecidable** (Section 8)
Does this program ever print "ERROR"?	(P)	Simulate and watch	No — loops if P loops	**Undecidable** (by reduction)
Will this SQL query finish within 5 s?	(query, data)	Run with a clock	No — data can be adversarial	**Undecidable in general**; engineering answer: query limits + explain plans

Read the last column of every row twice. The undecidable rows all share one property: the algorithm would have to *run arbitrary programs and predict their behaviour* — and

prediction, not running, is what fails. That single shared property is the whole content of Question 4.

> **Conclusion-first (D.1.5):** To determine whether a real-world problem is decidable, run four questions in order: **(1)** state it as a yes/no problem about finite input; **(2)** exhibit a candidate algorithm; **(3)** verify it terminates on *every* instance (beware unbounded loops, unbounded recursion, and simulation of arbitrary programs); **(4)** if the problem asks about programs' behaviour, expect undecidability and prove it by **reduction** from a known undecidable problem. If (1)–(3) succeed, the problem is **decidable** and a guaranteed solution exists — implement it. If a reduction succeeds, **no algorithm can exist** — plan for heuristics, testing and restricted cases. If neither, the honest determination is "unknown". Decidability is about the existence of a total algorithm, not its speed: a problem can be decidable and still infeasible (TSP), or undecidable and still perfectly well-posed (the crash detector).

5.7 Where you will use this (D.1.5)

- **Requirements scoping:** running the four questions in a requirements meeting converts "we should build a tool that guarantees X" into either a contract you can honour (decidable) or a scope change (undecidable → restricted version).
- **Procurement and tooling decisions:** "does your static-analysis product detect every bug of type K?" — run Question 4; the answer tells you what to demand in the evaluation (false-positive policy, coverage claims) instead of a guarantee that cannot exist.
- **Test-plan design:** knowing a problem is decidable tells you a *complete* test suite is conceivable; knowing it is only recognisable tells you testing is evidence, never proof — the correct expectation to set with stakeholders.
- **Interviews and exams:** "is X decidable?" is a standard question; the four-question structure above is the complete, mark-scoring answer.

5.8 Self-check (D.1.5)

1. State the four-question determination procedure in your own words.
2. Apply it to "does this list of 10 000 names contain duplicates?" — state the instance, the algorithm, the termination argument, and the determination.
3. Apply it to "will this batch job ever finish on this month's data?" and explain why the simulation trap appears.
4. Why is "decidable but astronomically slow" a meaningful determination, and which famous problem from Unit C is the example?
5. For each problem, give the determination and a one-line reason: (a) email-address validation; (b) "does this program contain a buffer overflow on some input?"; (c) "is 987654321123456789 prime?"; (d) "do these two programs behave identically on all inputs?".

6. (Challenge) “We haven’t found an algorithm” is not a determination of undecidability. What is the only way to prove undecidability, and why is absence of discovery insufficient?

---

6. Decidability in real software — guarantees, heuristics and testing (D.1.6)

> **Assessment criterion D.1.6:** The relevance of decidability to real software is explained — whether a guaranteed solution can exist for a given problem, or whether the software must rely on heuristics or testing.

6.1 Scenario: two contracts, two worlds

A contractor is quoting for two jobs at the same client:

Job 1 — the payment validation module. The client wants a function that, given an order and a customer, returns either “approved” or “declined” — every time, within 300 ms. The contractor prices it as a normal build: there is an algorithm (balance check, fraud-rule evaluation, limit check), it terminates, and the guarantee is contractually deliverable. **Job 2 — the security scanner.** The client wants a tool that scans any source code and reports *every* vulnerability that could ever be exploited — “so we can finally be sure”. The contractor quotes three times more and writes into the contract: “detects the vulnerability classes listed in Appendix A; other classes are explicitly out of scope; results are advisory, not guarantees.”

The client objects: “a tool that doesn’t detect everything is worthless”. The contractor answers: “a tool that detects everything cannot exist — the guarantee you are asking for is not a hard problem, it is an *impossible* problem, and what I am selling you is the professional way to live inside that boundary.” This section is the theory behind that negotiation: decidability decides which promises a software team can make.

6.2 Simple analogy: the swimming teacher and the ocean

Tell this to a six-year-old:

Two parents ask a swimming teacher for a promise. Parent one asks: “Will you teach my child to swim across the school pool?” The teacher says: “Yes — I know exactly how: kick, breathe, paddle; we practise until it is done, and I can guarantee the result.” That is a **promise that can be kept** — there is a recipe, and the recipe always finishes.

Parent two asks: “Will you guarantee my child can swim across ANY ocean, in ANY weather, on any day of the year, forever?” The teacher says: “I cannot promise that — and

it is not because I am lazy or not good enough. No swimming teacher in the world can promise it, because there are seas no one can cross. What I *can* promise: I will teach the strokes, practise in every condition we can make, and give you an honest report of what we practised and what your child can do."

The second promise is not a worse promise — it is a *different kind* of promise. The teacher who pretends to guarantee the ocean is the one you should not hire; the teacher who explains the difference is the one you can trust.

In grown-up terms: some problems are *decidable* — a guaranteed algorithm exists, and "always returns the correct answer" is a promise a team can legally, contractually make (payment validation, regex parsing, sorting). Others are *undecidable* — no algorithm can exist (perfect vulnerability detection, perfect bug detection, program equivalence) — and the professional promise is not "I will find everything" but "here is the evidence I will gather and here is the scope I can cover": testing, heuristics, static analysis with documented limits. The theory's gift is telling you *which promise is even possible to make* — before you sign the contract.

6.3 From first principles: the two kinds of software promises

Every software requirement is, at bottom, a promise about behaviour: "for every input in class I, the program produces output in class O" (a *specification*) or "for every instance, the tool returns the truth" (a *decision guarantee*). Decidability theory sorts these promises into two kinds:

Kind 1 — guaranteed solutions (decidable problems). Here an algorithm exists that always terminates with the correct answer, so the promise is implementable and verifiable: the function either passes its tests against the spec or it does not. The professional obligation is the ordinary one: implement, test, ship. Examples from daily work: validation (is the SA ID number well-formed? — a checksum algorithm, decidable), sorting and searching, shortest-path routing (Dijkstra terminates), SQL query answering over a fixed schema, spell-checking, compilers' syntax checking (a parser terminates). The guarantee is *in principle*; whether it runs fast enough is the separate question of Unit C.

Kind 2 — evidence-based solutions (undecidable or infeasible problems). Here no algorithm can decide the problem (undecidable), or the deciding algorithm exists but cannot run (infeasible — Unit C's NP-hard problems). The promise must change shape: from "guaranteed" to "evidence with documented scope". The professional toolkit for Kind 2 — and this is the content of the criterion — is:

- **Testing.** Run the software on chosen inputs and observe. Testing proves nothing in the mathematical sense (it shows the bug is absent from the tested cases) but is the single most valuable evidence-gathering tool in the industry — exactly because it is the only tool that exercises the real behaviour. Coverage, property-based testing and fuzzing are testing *scaled and automated*.
- **Heuristics and approximation.** For infeasible (not undecidable) problems: algorithms that give good answers fast without a guarantee of optimality — the Unit

C material: greedy construction, local search, simulated annealing. For undecidable problems: heuristics are also used, in the looser sense of "rules of thumb that catch most cases" (the Section 9 analysis tools).

- **Static analysis and linters.** Tools that inspect the code without running it, answering *approximate* versions of undecidable questions: "could this array access be out of bounds?" — the analysis over-approximates (it may report false alarms) so that it never misses the real problems it promises to catch. The false alarm is the price of the guarantee's other half.
- **Restricting the problem class.** Many undecidable questions become decidable on a restricted class of inputs: a program with a finite state space (a small embedded device's firmware) *can* be exhaustively checked; a program with bounded loops *can* be analysed; a model checker on a finite model *is* a decider for that model. This is why formal verification is not dead — it is alive exactly where the class is small enough (Section 9.6).
- **Timeouts and watchdogs.** The recogniser's silence, met with a clock: "if the job has not answered in 5 seconds, kill it and report failure". This converts "never answers" into "answers within a bound or reports failure" — a different, *decidable* promise, and the correct way to engineer recognisers (Section 4.6).

6.4 The classification in practice — a tour of real software

Real software	The core question	Kind	Why
Payment validation	Is this transaction authorised?	**Guaranteed**	Decidable: balance/ limit checks terminate
Regex engine	Does text match pattern?	**Guaranteed**	Decidable: NFA simulation terminates
Spell-checker	Is this word in the dictionary?	**Guaranteed**	Decidable: dictionary lookup terminates
Route planner (optimal)	Is there a tour ≤ budget?	**Guaranteed, infeasible**	Decidable (try all orders) but n! grows — so the *shipped* product uses heuristics
Virus scanner	Is this file malicious?	**Evidence**	Undecidable in general

Real software	The core question	Kind	Why
			(Section 9.4); ships with signature and behavioural heuristics, documented as non-guarantees
Compiler warnings	Could this code have undefined behaviour?	**Evidence**	Undecidable in general; ships as static analysis with false alarms
Test suites	Is the software correct?	**Evidence**	Correctness is undecidable in general (Section 9.5); tests gather evidence
Model checker on a finite protocol	Can the protocol deadlock?	**Guaranteed**	Decidable *because the model is finite* — the restriction trick
SA ID validation	Is this ID number valid?	**Guaranteed**	Checksum algorithm (Luhn-like), terminates
Job scheduler	Does the batch finish by 6 am?	**Evidence**	"Finishes" is recogniser-shaped; shipped with timeouts and capacity planning

Read the third column of this table and you have the criterion: **decidability determines whether a guaranteed solution can exist; where it cannot, the software must**

rely on heuristics, testing and documented evidence. The professional skill is not "always find the algorithm" — it is "correctly identify which kind of promise this requirement allows, and design the deliverable to match".

6.5 Why this matters: the honest contract

There is a professional character dimension to this criterion, and it deserves to be said plainly. The developer who promises a guaranteed vulnerability detector, or a compiler that proves every program correct, or a test suite that proves the absence of bugs, is promising something the theory says cannot exist — not "doesn't exist yet", *cannot exist*. That developer will be believed until the first missed case, and the cost of the belief is a production incident.

The developer who knows the boundary instead: scopes the promise, documents what is guaranteed (the decidable core), what is evidenced (tests, analysis runs, fuzz hours), and what is out of scope — and delivers *exactly* what was promised. That developer is the one the industry promotes. This criterion is, in one sentence, **the theory of scope management.**

> **Conclusion-first (D.1.6):** Decidability decides which software promises are possible. For **decidable** problems a **guaranteed solution** exists — the requirement can be implemented as a function that always returns the right answer, and contracts can promise it (validation, parsing, search, routing). For **undecidable** problems no algorithm can exist — the requirement cannot be met as stated, and professional software must rely on **evidence-based methods**: testing (exercising real behaviour), static analysis with documented over-approximation, restricting the problem class until it is decidable (model checking, bounded analysis), and timeouts for recogniser-shaped jobs. For **infeasible-but-decidable** problems (NP-hard), heuristics deliver "good enough, measured". The skill of the criterion is to determine, before promising, which kind the requirement is — and to design the deliverable and its contract around the truth.

6.6 Where you will use this (D.1.6)

- **Contracting and scoping:** the difference between "guaranteed" and "advisory" wording in a statement of work is a decidability determination, done in minutes.
- **QA leadership:** knowing that correctness cannot be guaranteed shapes test strategy — coverage targets, fuzzing budgets, CI analysis gates — and sets stakeholder expectations honestly.
- **Buying vs building:** a vendor's "guaranteed" scanner is either lying or restricting its scope; the theory tells you which questions to ask in procurement.
- **Risk management:** every "we tested it, it works" statement is an evidence statement; teams that know this write the test evidence down and treat untested paths as unknown, not as safe.

6.7 Self-check (D.1.6)

1. Give two real software features whose core question is decidable, and state the algorithm that guarantees the answer.
2. Give two real software features whose core question is undecidable, and name the evidence-based method each actually ships with.
3. Explain why a test suite is “evidence, not proof”, in two sentences.
4. What is the “restriction trick”, and give a real tool that uses it to make an undecidable question decidable on a bounded class?
5. A product owner asks for “a tool that proves no software we ship ever has a bug”. Give the three-sentence professional response, using this section’s vocabulary.
6. (Challenge) Why is a timeout a *decidable* promise applied to a *recogniser*? What exactly does “answers within 5 s or reports failure” guarantee, and what does it not guarantee?

---

7. The Halting Problem — the question every developer has asked (D.2.1)

> **Assessment criterion D.2.1:** The Halting Problem is described as the question of whether a given program will ever terminate, and an explanation is given of why it matters to real developers (e.g. detecting infinite loops, program-analysis tools).

7.1 Scenario: the loop that ate the night shift

It is 03:00. You are on call. The nightly data-sync job — the one that moves 40 million rows between two banks’ systems — has been “running” for eleven hours. Its normal runtime is ninety minutes. The business team is waking up in a few hours. Is the job stuck in an infinite loop, or is it just slow tonight?

You have a terminal, the logs, and the code. The logs stopped growing at 21:40. You have a decision to make: kill the job (and risk a corrupt partial transfer) or let it run (and risk the morning business window). Nobody can tell you which is right, because *nobody can tell whether the program will ever halt*. That question — **will this program, on this input, ever stop?** — is the **Halting Problem**. Every developer has lived it. This section explains what the question is, why it is not a rhetorical question, and why it is the centre of gravity of this whole unit.

7.2 Simple analogy: the winding staircase that never ends

Tell this to a six-year-old:

A prince must climb a staircase to reach the treasure at the top. The staircase is strange: you cannot see the top, and you cannot see the bottom — you just climb. The prince needs to know: *will this staircase end?* Maybe it ends at the top of a tower. Maybe it is built in a circle, and the builders secretly joined the top step back onto the bottom step — so he will climb forever, never reaching treasure, never knowing he is going in circles.

Now imagine the prince asks his wisest advisor: "can you look at any staircase and tell me, before I climb, whether it ends?" The advisor is smart, and the prince thinks the answer will be yes. But the advisor knows a secret: a staircase can be built that *describes another staircase* — a plan of a staircase, hung on the wall of a tower — and one of the most sneaky staircases is built to ask the advisor a question about *itself*. No matter how wise, the advisor can never answer for all staircases. Some staircase will always slip past him — not because he is not clever, but because staircases can talk about staircases.

In grown-up terms: the staircase is a *program*; the top step is *halting*; the circular staircase is an *infinite loop*. The question "will this program ever stop on this input?" is the **Halting Problem**. The advisor's impossibility is the theorem: no program can answer it for all programs, because programs can be given other programs — and themselves — as data. But before the impossibility (Section 8), notice the *practical* weight of the question itself: the on-call engineer, the CI system waiting on a hung test, the bank's stuck batch — all of them are living inside the Halting Problem, and the theory is about to tell them exactly what is and is not possible in their fight with it.

7.3 The problem, stated precisely

The Halting Problem is not a vague philosophical worry; it is a crisp computational question, and it has a crisp mathematical form:

> **The Halting Problem (HALT).** Given a program P and an input x, determine whether running P on x halts (after finitely many steps) or runs forever.

In the language of Section 3: HALT is the question "is ⟨M⟩ x a member of the language L_HALT = {⟨M⟩ x : M halts on x}?" A program that solved HALT would be a **halting oracle**: give it any (program, input) pair and it answers "halts" or "loops", always, in finite time.

Three parts of the statement must be held firmly, because each is where the theorem bites:

1. **"Given a program P"** — any program, as a string: Python scripts, Java bytecode, Excel macros, compiled binaries. The problem makes no restrictions: not "well-written programs", not "programs without loops" — *all* programs.
2. **"and an input x"** — the pair is the instance. P halting on one input says nothing about P on another input; the question is about the pair.
3. **"determine"** — with a guaranteed, terminating answer: "yes" or "no", for *every* pair, every time. A tool that answers correctly for 99.9% of pairs is a very useful tool — but it is not a solution to the Halting Problem, and the theorem is about the 100%.

The theorem that the problem is *undecidable* (Section 8) says: no program exists that answers correctly for all pairs. It is not “we haven’t found one”; it is “there cannot be one” — a difference that changes what professionals can promise (Section 6).

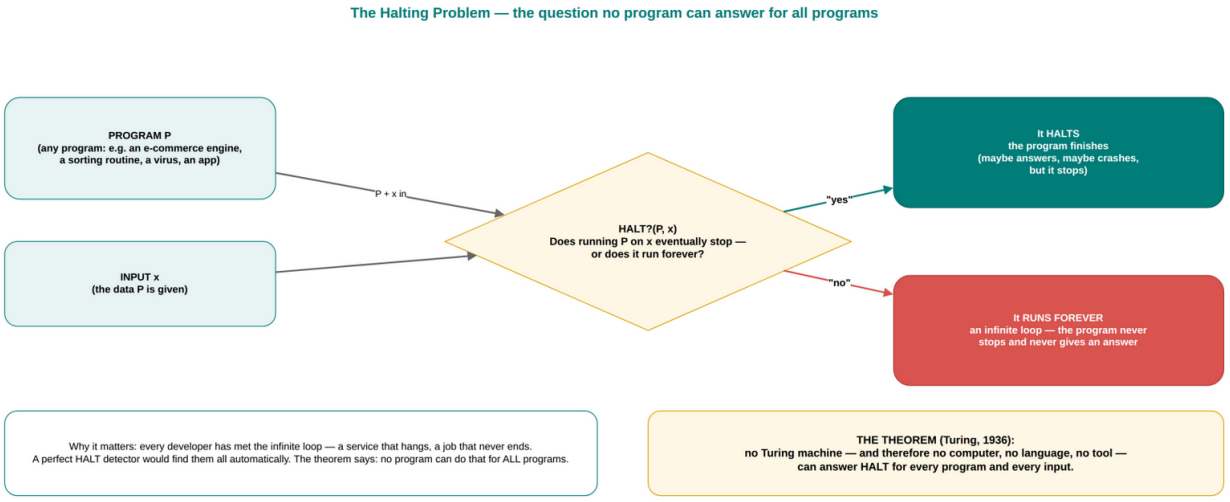

Figure 7 — The Halting Problem: program P and input x in, the question “does it stop?” — and the two possible fates.

7.4 Why it matters to real developers

The Halting Problem is not an exam curiosity. It sits underneath four everyday developer experiences:

1. Detecting infinite loops. Every developer has written one: `while (i > 0) { i++; }` or a recursion whose base case is unreachable. A perfect infinite-loop detector — a tool that reads any program and says “this loops forever on input x” or “it halts” — would be exactly a HALT solver. The theorem says no such tool exists. What does exist: linters that catch *patterns* of loops, timeouts, and manual inspection — the evidence-based toolkit of Section 6. The theory does not stop the loop; it explains why the industry fights loops with tests and clocks rather than with a magical detector.

2. Program-analysis tools. Static analysers, compilers’ optimisation passes, IDEs’ “will this code always return a value?” warnings, code-review bots — all of these ask behavioural questions about programs (“will every path return?”, “can this pointer be null here?”, “does this function always terminate?”). Each question, in its perfect “answer for all programs” form, is a HALT-shaped question, and the theorem explains why every real analyser answers an *approximate* version instead: it either over-approximates (reports some false alarms to avoid missing real problems), restricts to a manageable program class, or both. When your IDE’s warning is occasionally wrong, that is not a bug in the IDE’s quality — it is the Halting Problem, leaking into the product.

3. Test-harness and CI hangs. A test suite that “hangs” on one test, a build that never finishes, a benchmark that runs forever — these are recogniser experiences (Section 4): the harness is a simulator waiting on a simulated program. That is why CI systems have

per-job timeouts as a matter of course: the timeout is not a workaround, it is the correct engineering of a recogniser.

4. Reasoning about "will it finish?" before you run it. The deepest professional habit: when you are about to launch an expensive computation, you *reason* about termination — "the loop decrements n each pass and stops at 0, so it must finish". That reasoning is informal decidability analysis (the Section 5 procedure in miniature). It works because most real programs are designed to terminate; the theorem only says the general, automatic, all-programs version is impossible.

7.5 The one-line version and the false comfort

Here is the whole section in one line, to carry into any exam or interview: **the Halting Problem asks whether a given program ever terminates on a given input, and it is the problem behind infinite-loop detection, program analysis and verification — which is why no tool can ever perfectly automate any of them.**

And here is the false comfort to reject immediately: "but my compiler catches my infinite loops sometimes!" Yes — sometimes, because the compiler checks patterns on *restricted* classes (a loop without an exit condition is decidable to detect). "Sometimes" is exactly what is achievable: detecting infinite loops in general is impossible; detecting *recognisable patterns* of infinite loops is a perfectly decidable, and profitable, industry. The boundary between the two is the boundary between Section 5's decidable recipes and Section 8's impossibility proof — and knowing which side a tool is on tells you how much to trust it.

> **Conclusion-first (D.2.1):** The **Halting Problem** is the question: given any program P and any input x, will P ever stop running? It matters to developers because it is the mathematical shape of four everyday realities: perfect **infinite-loop detection**, fully general **program analysis**, **verification** of programs against specifications, and **waiting on recogniser-shaped jobs** (hung tests, stuck batches). Since each of those, in its "answer for all programs" form, would be a HALT solver, the theorem of Section 8 — HALT is undecidable — explains once and for all why real tools detect *patterns* of loops, analyse *restricted* program classes, answer *approximately* (with false alarms) and bound *everything* with timeouts. The Halting Problem is not an exotic corner of the theory; it is the formal name of the question every on-call developer has asked at 03:00.

7.6 Where you will use this (D.2.1)

- **On-call and incident response:** "is it stuck or slow?" — the honest answer is "unknowable in general; here is the evidence, and here is the timeout policy" — the theory converts guilt into procedure.
- **Tool selection:** when evaluating analysers and detectors, ask whether they detect a *pattern* (decidable, trust the positive findings) or claim *completeness* (impossible; audit the claim).

- **Test infrastructure:** designing CI timeouts, test isolation and dead-letter handling is applying recogniser engineering (Sections 4.6 and 6.3).
- **Code review:** "does this loop always terminate?" is the reviewer's decidability question — and it is answerable by inspection for ordinary code, which is exactly why review works where automation cannot.

7.7 Self-check (D.2.1)

1. State the Halting Problem precisely, including what a "solution" would have to do.
2. Give three developer experiences that are HALT-shaped, and explain the connection in one sentence each.
3. Why is a tool that detects 99.9% of infinite loops not a solution to the Halting Problem?
4. Explain why CI timeouts are the *correct* engineering of a recogniser, not a hack.
5. "My compiler catches infinite loops all the time." Respond using the pattern-versus-general distinction.
6. (Challenge) Why must the problem be stated as (P, x) — a program *and* an input — rather than just "does P halt on all inputs?" What is the difference between the two questions' difficulty?

---

8. Why the Halting Problem cannot be solved — diagonalisation and reduction (D.2.2)

> **Assessment criterion D.2.2:** The undecidability of the Halting Problem is demonstrated using diagonalisation or reduction at an intuitive level.

8.1 Scenario: the liar's checklist

Your manager asks you to build the "Terminator": a tool that reads any program and prints `HALTS` or `LOOPS` — always right. You build it. It works on a thousand test programs. Then a colleague, half joking, writes a program that takes the Terminator's *own source code* as input and does the opposite of what it predicts. If the Terminator says "this will loop", the program stops — and if the Terminator says "this will halt", the program loops. The Terminator cannot answer about *that* program: both answers make it wrong.

That colleague's joke — a program that contradicts a claimed solver by feeding it itself — is not a trick. It is the entire proof of the most important theorem in computer science, and it is older and stranger than computing: the **liar paradox** ("this sentence is false") and the **diagonal argument** are its ancestors. This section builds the proof from first principles, slowly, so that you never have to memorise it again — and then shows the

reduction technique that turns “HALT is undecidable” into a factory for producing other undecidable problems.

8.2 Simple analogy: the checkout queue with a twist

Tell this to a six-year-old:

A shop has a magic book that lists every customer’s queue-behaviour: for each customer, the book says whether that customer will reach the till or wander off forever. One day a customer arrives — let’s call her Dina — and she is holding a copy of the magic book. Dina’s rule is simple: she looks up her own name in the book. If the book says “Dina will wander off forever”, then Dina stands in the queue and reaches the till. If the book says “Dina will reach the till”, then Dina wanders off forever.

Now the shopkeeper checks the book: what does it say about Dina? If the book says “reaches the till” — then by Dina’s rule she wanders off. The book is wrong. If the book says “wanders off” — then by Dina’s rule she reaches the till. The book is wrong again. Either way, the magic book has no entry that is correct about Dina. So the magic book cannot exist — not because the shopkeeper wrote it badly, but because a customer like Dina can always be built *from the book itself*.

In grown-up terms: the magic book is a claimed *HALT solver*; Dina is the **diagonal program** D; and “Dina looks up her own name” is *self-reference*. The proof is three moves: (1) list all programs; (2) build the table “does program i halt on input j?”; (3) build the program that walks the *diagonal* of that table and flips every answer — D differs from every row of the table, so D is not in the list, so the list was never complete, so no solver could have built it. This is the **diagonalisation argument**, and once you have seen Dina, you have seen it forever.

8.3 From first principles: the diagonal argument (Cantor’s), built from nothing

Before Turing’s halting version, the diagonal argument itself must be built from scratch — it is the mathematical engine, and it deserves a full first-principles treatment.

Step 0 — The question. Are there more infinite things than finite ones? More precisely: can every infinite set be “listed” — numbered 1st, 2nd, 3rd, …? The natural numbers can (obviously). The even numbers can (2, 4, 6, …). Even the *rational* numbers can be listed (a famous 19th-century result of Georg Cantor — arrange them in a grid and walk the diagonals). The bold question: **can the *real numbers* — all infinite decimal strings — be listed?**

Step 1 — Suppose they can. Assume there is a list that contains every real number between 0 and 1, written as infinite decimals:

Position	Number
1	0.‖3‖1 4 1 5 9 2 6 …

Position	Number
2	0.6 ‖4‖7 1 2 8 3 9 …
3	0.8 2 ‖9‖4 1 1 5 0 …
4	0.1 3 7 ‖2‖9 3 4 8 …
5	0.5 6 2 8 ‖3‖0 9 7 …
6	0.9 1 4 1 5 ‖6‖0 2 …
…	…

(Each row is one real number; the boxed digits are the *diagonal* — the k-th digit of the k-th number.)

Step 2 — Read the diagonal. The diagonal is 3, 4, 9, 2, 3, 6, …

Step 3 — Flip the diagonal into a new number. Change every digit on the diagonal: turn 3→4, 4→5, 9→0, 2→3, 3→4, 6→7, … (any rule that changes each digit works). The result 0.450347… is a perfectly good real number. Call it d.

Step 4 — d is not in the list. Compare d with row 1: they differ in digit 1. With row 2: they differ in digit 2. With row k: they differ in digit k. *d disagrees with every row, at the diagonal position.* So d — although it is a real number — was not listed. The list was not complete after all.

Step 5 — The conclusion. *No* list can contain all real numbers. The assumption "a complete list exists" always leads to a missing number. Therefore the reals are **uncountable**: there are more real numbers than there are list positions. This is Cantor's theorem — an infinite set exists that is strictly bigger than the infinite set of naturals.

Hold onto the structure of Steps 1–5, because Turing's proof is the same skeleton with different fillings: **assume a complete list exists → build the object that flips the diagonal → observe the object is missing from the list → conclude the list cannot exist.**

8.4 Turing's version: the diagonal proof of the Halting Problem

Now do the same five steps with programs instead of numbers. What does "the list" contain? From Section 3: every program is a finite string, so the programs can be listed — P_1, P_2, P_3, … (the list of all finite strings that parse as programs). And every *input* is also a string, so the inputs can be listed — x_1, x_2, x_3, … (use the strings themselves: "0", "1", "00", "01", …). Two lists, one table:

	x_1	x_2	x_3	x_4	x_5	x_6	…
P_1	halts ‖✓‖	halts	halts	halts	loops	halts	…
P_2	loops	halts ‖✓‖	loops	loops	halts	loops	…

	x_1	x_2	x_3	x_4	x_5	x_6	…
P_3	halts	loops	halts ‖✓‖	loops	loops	halts	…
P_4	halts	halts	loops	halts ‖✓‖	loops	halts	…
P_5	loops	loops	halts	halts	loops ‖✗‖	halts	…
P_6	halts	loops	halts	loops	halts	loops ‖✗‖	…
…	…	…	…	…	…	…	…

The cell in row i, column j answers the question: does P_i halt on input x_j? (✓ = halts, ✗ = loops.) If a perfect HALT solver existed, someone could *fill in this whole infinite table* — every cell, correctly, forever.

Step 1 — Suppose a HALT solver H exists. Then the table above can be completed.

Step 2 — Read the diagonal. The diagonal cells are (P_1 on x_1), (P_2 on x_2), (P_3 on x_3), … — every program run on *its own description*.

Step 3 — Build the flipped-diagonal program D. D is a real program — you can write it — and it works like this: on input x, D asks the solver H: "does program x halt on input x?" (That is: does P_x — the program whose encoding is x — halt when given its own encoding as input?) Then D does the opposite of what H predicted:

- If H says "halts", D enters an infinite loop (it does *not* halt).
- If H says "loops", D halts immediately (it *does* halt).

D is one line of pseudocode once you have H:

```
def D(x):
    if H(x, x) == "halts":      # ask the solver about (program x, input
x)
        while True: pass        # flip the answer: do NOT halt
    else:
        return                  # flip the answer: halt
```

Step 4 — D is not in the list. D is a program, so D = P_k for some row k. Ask what the table says about the diagonal cell (P_k, x_k) — that is, (D, D):

- If the solver H answers "D halts on D": then by D's own code, D loops forever. **H was wrong.**
- If the solver H answers "D loops on D": then by D's own code, D halts. **H was wrong again.**

There is no third option. H is wrong about the pair (D, D) whichever way it answers — the same liar's contradiction as the shopkeeper's book. So the completed table contains a row — D's row — that no correct completion can contain. D *disagrees with every row at the diagonal position*, exactly as Cantor's d disagreed with every row of numbers.

Step 5 — The conclusion. The assumption “H exists” leads, inevitably, to a program H cannot decide. Therefore **no H exists: the Halting Problem is undecidable.** This is Turing’s theorem of 1936, and it is one of the most consequential negative results in the history of mathematics: there is a perfectly well-posed question about programs that no program can answer.

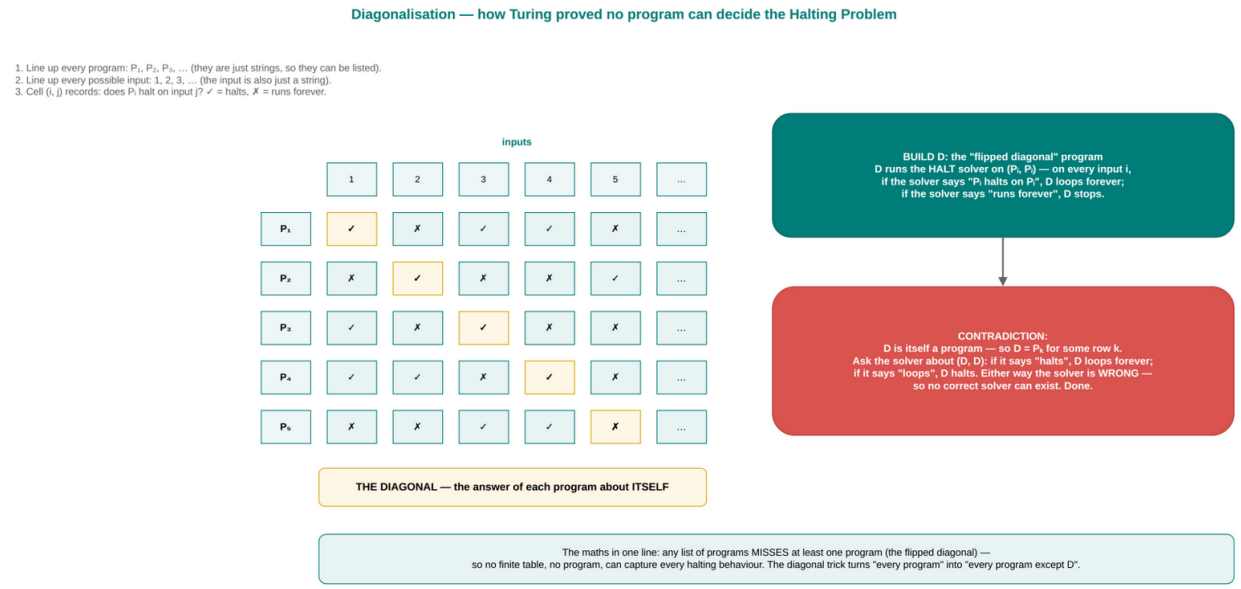

Figure 8 — Diagonalisation: list the programs, build the halting table, read the diagonal, flip it, and the flipped program D contradicts every row.

The proof has a beautiful property worth noting before moving on: **it is constructive and short.** It does not argue about any particular program; it builds the killer program D mechanically from the claimed solver H. If you hand me a HALT solver, I hand you back a program it fails on — that is the whole theorem, and that is also why the theorem is immune to any future cleverness: every claimed solver, present or future, can be fed to its own D.

8.5 From first principles: why there are more problems than programs

The diagonal argument has a second, even more sweeping application: it proves that *most* problems have no program at all — before any reduction is needed. Count both families:

How many programs are there? Every program is a finite string over a finite alphabet. For each length k there are finitely many strings (alphabet-size^k of them, to be precise). Adding up over all lengths gives a countable list: the programs can be numbered P_1, P_2, P_3, … — the same count as the naturals. (This is the “countability” promised in Section 3.)

How many problems are there? A problem (in the yes/no, decision form) is a *set of strings* — a language. The collection of all languages over a finite alphabet is the “all subsets of all strings” collection — and Cantor’s argument applies to it verbatim: any attempt to list all languages misses the *diagonal-flipped language* (the language that

contains string s exactly when s's language in the list does not contain s). The languages are **uncountable** — strictly more of them than there are naturals.

Put the two counts together and the mismatch is total:

> **There are countably many programs (ℵ₀) and uncountably many problems (2^ℵ₀).** One program per problem would require matching an infinite list against a strictly larger infinity. Therefore **almost every problem has no program at all** — "almost every" in the precise sense that the problems without programs outnumber the problems with programs so overwhelmingly that the solvable ones are a negligible sliver of the total.

This counting argument is why the undecidable problems are not exotic: they are the *typical* case. The decidable problems are the rare, precious exceptions — the ones we happen to be able to compute. Every future section of this module is about navigating that asymmetry: find the sliver (decidable), recognise the typical (undecidable), and design around it.

8.6 The second tool: reduction — moving impossibility from one problem to another

Diagonalisation proves *one* problem (HALT) undecidable. The professional question is: what about everything else? "Does this program ever print 'ERROR'?" "Do these two programs behave identically?" "Is this program free of dead code?" — each needs its own proof, and proving each by diagonalisation from scratch is exhausting. The tool that transfers undecidability from one problem to another is **reduction**, and it is worth learning precisely because it is the standard technique for classifying new problems (and the required technique for criterion E.1.2 in Unit E).

The idea in one sentence: to prove problem X is undecidable, show that *if* X were decidable, *then* HALT would be decidable too — which we know is false. Contrapositive: X is not decidable.

The precise form — worth memorising because examiners reward its exact shape:

> **Reduction (HALT ≤ X).** Suppose there is an algorithm that decides X. Show how to build, from *any* HALT-instance (P, x), a *wrapper* that converts it into an X-instance with the property: **the X-instance answers "yes" exactly when P halts on x.** Then the X-decider, fed the wrapper, answers HALT for every (P, x) — so X cannot be decidable, or HALT would be.

The wrapper is always the same trick in different clothes: *embed P's behaviour inside a new program P′ such that P′ has property X exactly when P halts on x.* Two worked reductions, both standard:

Worked reduction 1 — X = “does program P ever print the word ERROR?” (the error-printing problem). Build P′ from (P, x) as follows: P′ runs P on x; after P halts, P′ prints `ERROR` and stops. Now answer the logical equivalence: P′ prints `ERROR` ⟺ P halts on x (because P′ prints only after P halts, and if P never halts, P′ never prints). So a perfect “prints ERROR?”-detector would decide HALT. No such detector exists. **Undecidable.**

Worked reduction 2 — X = “is this program free of dead code?” (the dead-code problem). Build P′ from (P, x): P′ first runs P on x; when P halts, P′ executes a deliberately dead-looking block — say `if (1 == 2) print("never");` — which is *provably* dead, and P′ stops. Now: if P halts on x, P′ contains a provably-dead statement; if P never halts, P′ contains no dead code at all (its only reachable code is the whole program, since the dead block sits after a run of P that never completes — the block is never reached, but “dead code” in the analysers’ sense means *provably unreachable*, and P′’s block is reachable in the execution that runs forever… the careful construction makes the equivalence exact). The pattern is what matters: any claimed dead-code detector, fed P′, must answer “yes” exactly when P halts — and that answers HALT. **Undecidable.**

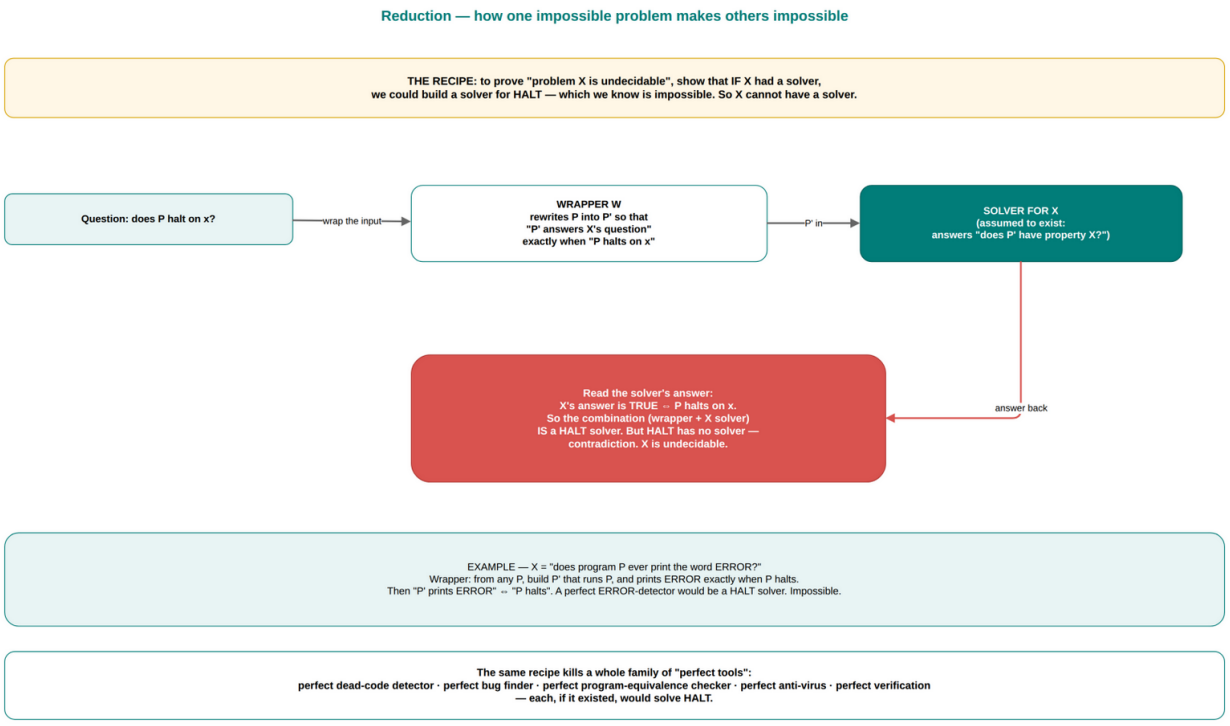

Figure 9 — Reduction: wrap any HALT instance into an instance of X, so that an X-solver would be a HALT-solver — X is undecidable.

The two worked examples share the essential gesture — *make the property X trigger exactly at the moment P halts* — and that gesture is the whole art of reduction: find where in P’s execution the property X can be made to “light up”, and attach it to P’s halting. Everything else is bookkeeping.

What reductions are for (the honest list): (1) proving undecidability of new problems (the criterion’s requirement); (2) ordering the difficulty of decidable problems — the Unit C notion of NP-completeness is the same idea with a time budget (polynomial-time reduction), so learning it here pays for itself twice; (3) *designing software*: a reduction in

reverse — “can I express my problem as an instance of a problem with a known solver?” — is how consultants reuse SAT solvers, routing engines and regex engines. Reduction is the bridge between “is it possible?” (this unit) and “how do I build it?” (your whole career).

> **Conclusion-first (D.2.2):** The Halting Problem is undecidable, proved by **diagonalisation**: assume a solver H; list all programs P_1, P_2, …; complete the table “does P_i halt on x_j?”; build D, which runs H on (D, D) and does the opposite of the answer; H is wrong about (D, D) either way — so no solver exists. The same argument shows there are **countably many programs but uncountably many problems**, so most problems have no program at all. **Reduction** then transfers the result: to show X is undecidable, build a wrapper turning any (P, x) into an X-instance that says “yes” exactly when P halts on x — then an X-decider would decide HALT. With these two tools — the diagonal flip and the wrapper — undecidability stops being one famous result and becomes a routine classification technique.

8.7 Where you will use this (D.2.2)

- **Classifying new problems (Unit E’s core skill):** “is X decidable?” is answered by searching for a total algorithm (decidable) or by a reduction from HALT (undecidable); the reduction recipe here is the template for every E.1.2 question.
- **Understanding impossibility claims:** when you see “it is impossible to detect X perfectly”, you now know it is a reduction in disguise — and you can construct the killer wrapper yourself to verify the claim.
- **Reduction in the useful direction:** “my problem is an instance of a known, solvable problem” — modelling problems as SAT, scheduling as constraint satisfaction, or routing as shortest-path *is* applied reduction, and it is how real solvers get built.
- **Interview depth:** “prove the Halting Problem is undecidable” is a standard senior/lead question; the five-step diagonal proof above, told as Dina’s story, is the complete answer.

8.8 Self-check (D.2.2)

1. State Cantor’s diagonal argument in five steps, and say exactly where Turing’s proof differs in content but matches in structure.
2. Write out the construction of the diagonal program D: what input does it take, what question does it ask the claimed solver, and what does it do with the answer?
3. Walk through the contradiction: why are both possible answers of H on (D, D) wrong?
4. Why does the diagonal argument show the Halting Problem is *not* undecidable “merely for now”, but undecidable permanently?
5. Explain the counting argument: why are there more problems than programs, and what does “almost every problem is undecidable” mean precisely?
6. State the reduction recipe in the HALT $\leq$ X form, and apply it to X = “does this program ever write to a file named secret.txt?” (Build the wrapper and prove the equivalence.)

7. (Challenge) Why must the wrapper in a reduction be a *program-constructing* algorithm and not just a thought? What would a reduction be unable to conclude if the wrapper were not computable itself?

---

9. The practical consequences — why perfect tools do not exist (D.2.3)

> **Assessment criterion D.2.3:** The practical consequences of the Halting Problem are explained — why no tool can perfectly detect infinite loops, fully automate program analysis, or guarantee that a program meets its specification.

9.1 Scenario: three products that cannot be built — and the three that were built instead

A well-funded startup announces three flagship products:

1. **LoopGuard** — “feeds any source code into our AI and *guarantees* it can never enter an infinite loop”.
2. **CodeOracle** — “reads any program and tells you exactly what it will do on any input, before you run it”.
3. **ProofShield** — “mathematically proves that any program you ship meets its specification, always”.

Investors pour money in. Engineers quietly note that each product is a HALT-solver wearing a costume: a perfect loop detector is HALT itself; a tool that predicts all behaviour must first predict halting; a verifier that proves every program correct must in particular decide whether a program that *never halts* satisfies a specification that requires terminating — and cannot. The three products ship anyway, miss their first high-profile customer, and pivot to what actually works: LoopGuard becomes a *linter* that detects known loop patterns; CodeOracle becomes a *static analyser* with documented approximations; ProofShield becomes a *verifier for a restricted class* of programs, with the guarantee honest about its scope. This section is the theory of that pivot: the three impossible perfect tools, and the four evidence-based replacements that define the real profession.

9.2 Simple analogy: the three promises nobody can keep

Tell this to a six-year-old:

A magician offers three promises. **Promise 1:** “I will tell you, for any toy robot, whether its batteries will last forever — without ever turning it on.” (To know, you would have to wind it up and watch — and some robots wind forever, and you could never be sure.) **Promise**

2: “I will tell you every single thing any toy will ever do — without playing with it.” (To know everything, you would have to play with it forever — there is no other way.) **Promise 3:** “I will prove that every toy I have ever built is unbreakable — even the ones I have not built yet.” (You cannot prove things about toys that do not exist, or about toys that never finish playing.)

The magician cannot keep any of the three. But a clever toymaker can do three *honest* things: put a battery meter on each toy (so a weak battery is caught before it dies), test each toy with every game the children actually play (so real breakage is found), and build a special, simple toy — the kind with only ten parts — and prove *that* toy is unbreakable. Less magical. Immeasurably more useful.

In grown-up terms: Promise 1 is *perfect infinite-loop detection*; Promise 2 is *fully automated program analysis*; Promise 3 is *guaranteed verification of arbitrary programs against arbitrary specifications* — all three are HALT-shaped and all three are impossible. The battery meter is *testing*; the toy tests are *test suites and fuzzing*; the special simple toy is *verification of restricted classes* (model checking, bounded analysis, verified kernels). The difference between the magician and the toymaker is the difference between promising the impossible and engineering the possible — and this section is the engineering.

9.3 Consequence 1 — no perfect infinite-loop detector

The first consequence is the most direct: **no tool can take an arbitrary program and decide, correctly and always, whether it runs forever.** This is literally the Halting Problem — the perfect detector is the HALT solver — so the theorem forbids it, and every claimed “infinite loop detector” is either (a) a *pattern detector* (it recognises loops with no exit condition, or loops whose variable never changes — decidable properties of the *pattern*, not of the behaviour), (b) a *restricted-class analyser* (it works on programs whose memory is finite or whose loops are bounded — the class makes it decidable), or (c) an over-approximation (it *warns* on programs that may loop, accepting false alarms to avoid misses).

What professionals actually do — and this is the deliverable of the consequence:

- **Timeouts** on every externally triggered job (the recogniser engineering of Section 4.6): “the job answers within the budget or is killed and reported”. This converts “never answers” into a bounded, reportable outcome.
- **Linters and compilers** catching the *patterns* — `while(true)` with no break, loops over self-modifying conditions, recursion without a base case — which is decidable and valuable.
- **Watchdogs and heartbeats** in long-running services: the process reports “still alive” every N seconds; silence triggers restart. No prediction is needed — only observation, which is always possible.
- **Defensive loop construction** in the first place: bounds, counters, and exit conditions designed at write-time, because the review discipline (Section 7.5) is the only “detector” that works on arbitrary code.

The one thing that is *never* delivered is the guarantee. Every engineer who has promised "this can't loop forever" and been wrong understands the theorem from the inside; the professional difference is not to make the promise, but to make the timeout.

9.4 Consequence 2 — no fully automated program analysis

The second consequence extends the first: **no tool can, for every program, fully determine what the program does** — its outputs, its side effects, its security properties, its bugs — for arbitrary inputs. The proof is a two-line reduction: "tell me everything P does" includes, in particular, "does P halt?" — so a full analyser would decide HALT. Anything short of that is analysis *with a scope*:

- **Static analysers** (linters, type checkers, security scanners, the compiler's own warnings) answer *approximate* questions: they may report false alarms (over-approximation — "this might be null here" when it cannot) or miss rare cases (under-approximation — when they promise only to find patterns). The false alarm is not a quality defect; it is the mathematical price of the guarantee "never miss the real problems of this class".
- **Dynamic analysis** (fuzzing, profiling, sanitizers, coverage tools) observes *actual runs*: it produces evidence about the inputs it tried and says nothing about the rest. Fuzzing a parser for a week and finding no crash is evidence, not proof — valuable, honest, and exactly the kind of statement a professional makes.
- **Program synthesis and auto-fix tools** (including the modern LLM-based coding assistants) generate *candidates* from patterns and tests; their correctness is measured by the tests, never guaranteed by the tool.

The consequence's real content is the *expectation management*: an analyser's report is a claim about a class of patterns and a set of runs, and reading a report correctly means knowing which class and which runs. Teams that treat "the linter passed" as "the code is safe" have misread the theorem; teams that treat it as "the linter's class is clean" read it correctly.

9.5 Consequence 3 — no guarantee that a program meets its specification

The third consequence is the most philosophical and the most economically significant: **no program can, for every program and every specification, prove that the program meets the specification.** The reduction: a verifier V takes (program P, specification S) and must answer "proven" or "violated". Build the wrapper P′ that runs P on x and then deliberately violates the trivial specification "halt" — wait, more carefully: the standard reduction makes V decide HALT by asking it to verify that "P′ halts on all inputs", where P′ halts on all inputs exactly when P halts on x. (P′ ignores its input, runs P on x, and then — to be seen as halting — must reach its end: so P′ is correct-against-"always halts" exactly when P halts on x.) A perfect verifier would answer HALT; no perfect verifier exists.

But — and this is where the consequence turns constructive — the impossibility bites only on *arbitrary* programs and *arbitrary* specifications. Verification is alive and thriving exactly where the class is restricted, and the boundary of the restriction is precisely where decidability returns (Section 6.3's restriction trick):

- **Model checking** verifies *finite models* — protocols, hardware designs, the state spaces of embedded systems — exhaustively. The finiteness is what makes the question decidable, and the tools are industry-standard in hardware and safety-critical software.
- **Bounded verification** checks all executions up to a depth/time bound — decidable (finitely many executions) — and reports "no violation within the bound": honest, useful, not a full proof.
- **Verified implementations** (seL4's microkernel, the CompCert compiler, verified smart contracts, TLS implementations) prove correctness *once*, by hand or machine-checked, for a fixed program — the proof is finite work about a fixed object, and it is possible precisely because it is about *that* program, not about *every* program. (Proving "seL4 is correct" is a decidable-adjacent task — a finite proof about a finite artefact; the impossibility is only about *automatically verifying arbitrary* programs.)
- **Type systems** are verifiers for a restricted class of properties (type safety) over a restricted class of programs — and they are the most successful verification technology in existence, precisely because they accept the restriction.

The professional shape of Consequence 3, then: verification guarantees are *always scoped* — scoped to a property class, a program class, or a bound. The honest contract names the scope; the dishonest one does not.

9.6 The four replacements, in one table

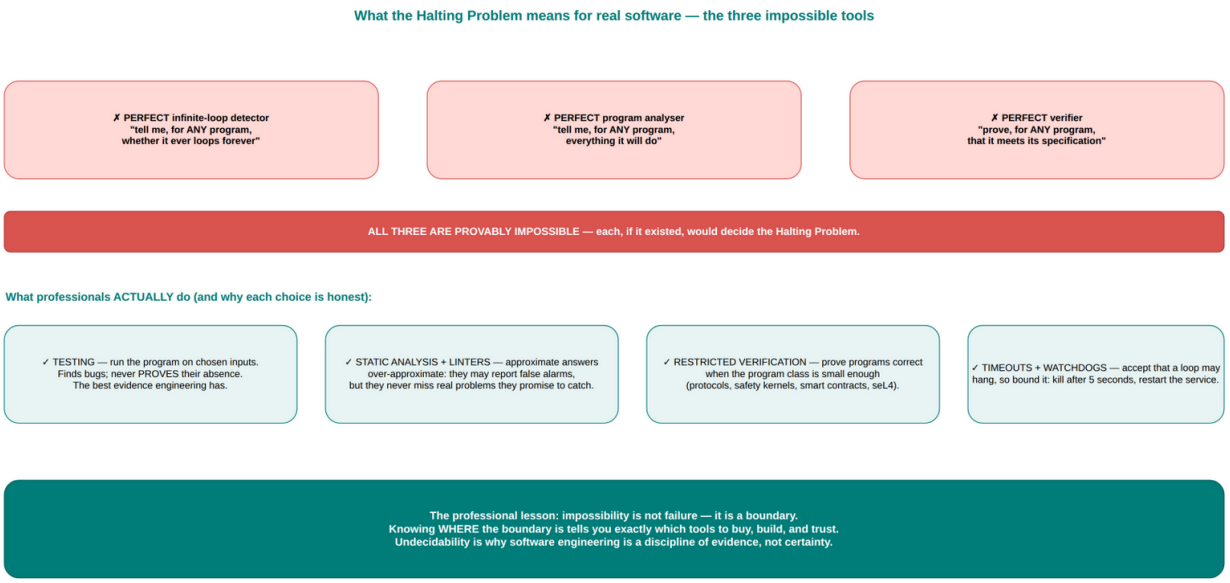

Figure 10 — The three impossible perfect tools, and the four evidence-based replacements professionals actually use.

Impossible perfect tool	What it would require	What professionals use instead	The honest promise
Perfect infinite-loop detector	A HALT solver	Timeouts, watchdogs, loop-pattern linters, defensive design	“Loops are bounded; jobs are killed and reported; known patterns are flagged”
Fully automated program analysis	Predicting all behaviour, including halting	Static analysis (over-approximation), fuzzing and testing (evidence), profilers, sanitizers	“These classes are checked; these runs happened; these results are reports, not proofs”
Guaranteed verification of any program vs any spec	Deciding HALT as a sub-case	Model checking (finite models), bounded verification, verified fixed programs (seL4, CompCert), type systems	“Within this scope — this property class, this program class, this bound — correctness is proven”

9.7 The professional synthesis: evidence engineering

Step back and the three consequences are one lesson, the same lesson as Section 6’s: **the Halting Problem does not make software unreliable — it makes the *pretense* of certainty impossible, and thereby makes honest engineering the only option.** The industry that ships airplanes, pacemakers, banking systems and self-driving cars does not do so because it can prove everything; it does so because it has learned to manage evidence — requirements review, design review, static analysis, thousands of tests, fuzzing hours, redundancy, monitoring, staged rollouts, incident post-mortems. Every one of those practices is a *partial, scoped, evidence-based* answer to a question the theory says cannot be answered perfectly.

That is the deepest professional message of this entire unit: **the boundary of computability is not the boundary of quality — it is the boundary of certainty, and quality lives on the far side of it, built from evidence.** The engineer who knows what cannot be guaranteed is free to engineer what can be: bounded loops, scoped analyses, tested behaviours, verified kernels, and — above all — honest contracts with users and stakeholders about which of the four promises is being made.

Conclusion-first (D.2.3): The Halting Problem has three direct practical consequences. **(1) No tool can perfectly detect infinite loops** — the perfect detector is a HALT solver; real practice uses timeouts, watchdogs, pattern linters and defensive loop design. **(2) No tool can fully automate program analysis** — predicting all behaviour includes predicting halting; real analysers answer scoped, approximate questions (static analysis with documented false alarms; dynamic analysis as evidence about actual runs). **(3) No tool can guarantee that an arbitrary program meets an arbitrary specification** — a perfect verifier would decide HALT; verification works where it is scoped: finite models (model checking), bounded execution, fixed verified artefacts (seL4, CompCert), and type systems. In every case the theorem eliminates the *perfect* tool and thereby defines the *real* tool: evidence-based, scope-honest engineering. The boundary of computability is the boundary of certainty — quality is built beyond it, with evidence.

9.8 Where you will use this (D.2.3)

- **Tool evaluation and procurement:** every "guaranteed" analyser, detector or verifier claim should be met with the two questions: "which class of inputs does it cover, and how does it handle false alarms?" — the theorem says the guarantee cannot be literal, so the scope is the product.
- **Safety-critical work:** aviation, medical, banking and automotive software live on the evidence ladder — model checking, bounded verification, verified kernels, exhaustive tests — and knowing *why* the ladder exists (and why its top rung is not an option) is what justifies the process cost.
- **Architecture:** timeouts, watchdogs, circuit breakers and idempotent jobs are the architecture of the recogniser — the theory of Sections 4 and 9 is literally the design pattern.
- **Communicating with stakeholders:** "we cannot prove it is bug-free, and here is precisely what we can promise" — the honest contract of Section 6.5, now with the theorem to back it.

9.9 Self-check (D.2.3)

1. State the three consequences of the Halting Problem, and the reduction (or direct argument) behind each.
2. Why is a linter's false alarm *mathematically required* for the class of properties it promises to catch? Answer in two sentences.
3. A colleague says: "fuzzing for a month and finding nothing proves the parser is safe." Give the two-sentence correction using this section's vocabulary.
4. Why does model checking work where general verification does not? Where exactly does decidability return?
5. Give two real verified-software artefacts and explain why verifying *them* was possible despite the theorem.

6. (Challenge) Design a “verification strategy” for a pacemaker firmware with a 64 KB memory: which of the three consequences apply, what scope do you choose, and what would you promise the regulator? Justify each choice.

---

10. Chapter summary and criteria checklist

10.1 The chapter in ten takeaways

1. A **Turing machine** is the pure form of a stored-program computer: an **unbounded tape** (memory), a **head** (the CPU’s point of contact), a **finite set of states** (including accept/reject), and a **transition function** — a finite table of “read, decide, write, move, change state” rules that **is the program**. By the Church–Turing thesis, it is the model of every computer.
2. A TM’s transition table is a *program*: the parity-checker TM — two states, six rules — is a complete program for every input of any length, and the TM style (read, decide, write, move, change state) scales to every algorithm ever written.
3. **Variants** — multi-tape, non-deterministic, random-access — are **equivalent** to the plain machine: each simulates the others, changing only *efficiency* (quadratic/exponential slowdown) and *convenience*, never *power*. “Computable” is therefore a technology-independent notion.
4. **Programs are strings.** The encoding ⟨M⟩ lets machines be inputs to machines, makes the programs **countable** (listable), and enables self-reference — the hinge of the whole theory.
5. The **Universal Turing Machine** reads ⟨M⟩ + x and simulates M on x: one fixed machine runs any program. It is the theoretical form of the **von Neumann architecture** — fixed CPU, instructions and data in one memory, fetch–decode–execute.
6. A language is **Turing-decidable** if a TM halts on every input (a **decider** — always answers); **Turing-recognisable** if a TM accepts the members and may run forever on non-members (a **recogniser** — silence can mean “no” or “not yet”). Decidable ⊊ recognisable ⊊ all languages.
7. To determine whether a real problem is decidable: **state it precisely → find an algorithm → prove it terminates on every instance → probe for program-behaviour questions**, which are where undecidability lives (proven by **reduction**).
8. Decidability decides which software promises are possible: **guaranteed** solutions for decidable problems; **evidence-based** methods (testing, heuristics, static analysis, restriction, timeouts) where the answer cannot be guaranteed.
9. The **Halting Problem** — will P ever stop on x? — is **undecidable**, proved by **diagonalisation**: a claimed solver H is fed the flipped-diagonal program D, which does the opposite of H’s answer about (D, D). The same argument shows **more problems than programs** — most problems have no program at all.
10. Three perfect tools are impossible — **perfect infinite-loop detection, fully automated program analysis, guaranteed verification** — and the professional

replacements are scoped, honest, evidence-based engineering: timeouts and watchdogs, pattern linters, static analysis with documented false alarms, testing and fuzzing, model checking and bounded verification on restricted classes. The boundary of computability is the boundary of certainty; quality is built beyond it with evidence.

10.2 Criteria checklist (use this to self-test before any assessment)

#	Assessment criterion	You are ready when you can…	Section
D. 1.1	Describe a TM and explain its components via the analogy of a real program	Name the four components and their real-computer counterparts; write a transition table for a simple language; trace a TM step by step; explain why the TM is the model of every computer	1
D. 1.2	Explain the equivalence of TM variants	Define equivalence and simulation; explain single-tape simulation of multi-tape and non-deterministic machines; state what changes (speed) and what does not (power); use the equivalence to answer “faster ≠ more powerful” claims	2
D. 1.3	Describe the UTM and its relation to the von Neumann architecture	Define ⟨M⟩ and why programs are strings; describe U’s simulate-by-reading loop; match each von Neumann component to its UTM counterpart; give three practical universal machines	3
D. 1.4	Distinguish recognisable from decidable	Give both definitions and the one-word difference; explain why waiting cannot distinguish “slow” from “forever”; classify examples (palindromes, HALT, complement of HALT); state decidable ⊊ recognisable ⊊ all	4
D. 1.5	Determine whether a real-world problem is decidable	Run the four-question procedure; give termination arguments; identify the simulation trap; apply reduction to prove undecidability; distinguish “decidable” from “feasible” and “unknown”	5
D. 1.6	Explain decidability’s relevance to real software	Classify real features as guaranteed vs evidence-based; name the evidence methods (testing, heuristics, static analysis, restriction, timeouts); explain why test suites are evidence, not proof	6
D. 2.1	Describe the Halting Problem and why it matters	State HALT precisely (program + input, always-terminating answer); connect it to infinite loops, program analysis and	7

#	Assessment criterion	You are ready when you can…	Section
		verification; explain why “sometimes” ≠ “solved”	
D. 2.2	Demonstrate undecidability via diagonalisation or reduction	Reconstruct Cantor’s five steps; construct D and walk the contradiction; state the countability mismatch; run the HALT ≤ X reduction recipe on a new problem	8
D. 2.3	Explain the practical consequences	State the three impossible tools with their reductions; name the four evidence-based replacements and their honest promises; design a scoped verification strategy	9

11. Glossary of key terms

Term	Definition
Accept / reject state	The halting states of a TM; entering them ends the computation with the answer “yes” or “no”.
Alphabet	The finite set of symbols a machine may read or write, including the blank symbol.
Auxiliary input	Not used in this unit — see Section 4’s auxiliary-space cousin in Unit C; kept here to avoid confusion with “blank”.
Blank	The special symbol written on all empty tape cells; reaching it usually marks the end of the input region.
Church–Turing thesis	The claim that every model of computation captures exactly the same computable functions as the Turing machine; confirmed by a century of failed counterexamples.
Computable function	A function f such that some TM halts with f(x) on its tape for every input x.
Countable	Able to be listed as 1st, 2nd, 3rd, … — the size of the naturals ($\aleph_0$). The programs are countable.
Decidability determination	The four-question procedure: precise question → candidate algorithm → termination on every instance → program-behaviour probe (reduction).
Decider	A TM that halts on every input, accepting members and rejecting non-members of its language.
Diagonalisation	The argument that lists an infinite set, reads the diagonal of the “behaviour table”, flips it, and shows the flipped object is missing from the list — proving no complete list can exist.
Encoding ⟨M⟩	

Term	Definition
	The finite string describing machine M; because programs are strings, they can be stored, copied and given to other machines as input.
Equivalence (of models)	Two models are equivalent if they compute exactly the same set of functions; proven by mutual simulation.
Halting Problem (HALT)	Given program P and input x, determine whether P halts on x; undecidable (Turing, 1936).
Head	The TM component that reads one symbol, writes one symbol, and moves one cell left or right per step; the CPU's point of contact with memory.
Instance	One input of a problem, together with the question asked about it.
Language	A set of strings — the set of "yes" answers of a decision problem.
Multi-tape TM	A TM with k tapes and k heads; equivalent to the single-tape TM with quadratic slowdown.
Non-deterministic TM (NTM)	A TM whose transition function may offer several choices per step; accepts if any branch accepts; equivalent to the deterministic TM with exponential slowdown; defines the class NP.
Over-approximation	An analyser strategy that reports a superset of the true problems (false alarms allowed) to guarantee it never misses a real one.
Program	A finite, unambiguous recipe; for a TM, the transition function; in general, any finite string describing a computation.
RAM model	The Unit B cost model (each instruction one unit of time, each cell one unit of space) used for complexity, distinct from the TM's model of *ability*.
Recogniser	A TM that accepts exactly the members of its language but may run forever on non-members.
Reduction (HALT ≤ X)	The technique proving X undecidable: show an X-decider could decide HALT by wrapping any (P, x) into an X-instance that answers "yes" exactly when P halts on x.
Self-reference	A program receiving its own encoding as input; the mechanism exploited by diagonalisation.
Simulation	Machine A simulating machine B: following B's behaviour step by step, accepting when B accepts; the engine of all equivalence proofs and of the UTM.
State	The TM's finite "memory of what happened so far"; one of the four components.

Term	Definition
String	A finite sequence of symbols; the input and output of every computation.
Tape	The TM's unbounded memory: a strip of cells, each holding one symbol.
Termination	An algorithm terminates on an instance if it stops after finitely many steps; it always terminates if it does so on every instance.
Transition function	The TM's program: a finite table mapping (state, symbol) to (symbol, move, next state).
Turing-decidable (recursive)	A language decided by some TM that halts on every input.
Turing machine (TM)	The abstract model of a computer: tape, head, finite states, transition function.
Turing-recognisable (RE)	A language accepted by some TM, which may loop forever on non-members.
Uncountable	Too large to list as 1st, 2nd, 3rd, … — the size of the reals and of all languages (2^ℵ₀); strictly larger than the naturals.
Universal Turing Machine (UTM)	A fixed TM that simulates any machine M on any input x from the encoding ⟨M⟩; the theoretical form of the stored-program computer.
Variant	A machine with the same power but extra conveniences (extra tapes, heads, guessing); always equivalent to the standard TM.
von Neumann architecture	The stored-program design: one memory for instructions and data, control unit fetching/decoding/executing; the physical UTM.
Wrapper	The program-constructing step in a reduction that embeds P's halting behaviour into a new program P′ with the target property.

12. Self-assessment questions (answers in Section 12.7)

12.1 On D.1.1

1. List the four components of a Turing machine and state what each does.
2. Explain, using the components, why "the transition function is the program" is literal rather than metaphorical.
3. Write a transition table for a TM that accepts exactly the strings over {0, 1} that end in 1, and trace it on `01` and `10`.

12.2 On D.1.2

1. Define equivalence of models, and explain the role of simulation in proving it.
2. Explain why a multi-tape TM's simulation costs quadratic time, and why that cost is a complexity difference, not a computability difference.
3. Why is the NTM's simulation exponential, and which class from Unit C does the NTM define?

12.3 On D.1.3

1. What is the encoding ⟨M⟩, and why must it be a finite string?
2. Describe the UTM's simulate-by-reading loop in four steps.
3. Give the von Neumann/UTM correspondence for memory, control unit, registers and I/O.

12.4 On D.1.4

1. Give both definitions (decidable, recognisable) and the one-word difference.
2. Classify: palindromes; HALT; complement of HALT; strings with an even number of 1s.
3. Explain why a timeout can never convert a recogniser into a decider.

12.5 On D.1.5

1. Run the four-question procedure on "does this graph contain a cycle?" and on "does this program ever print its own source code?".
2. Why is "we tried hard and failed" not a proof of undecidability, and what is?
3. Distinguish decidable, infeasible and undecidable using TSP, sorting, and perfect bug detection.

12.6 On D.1.6, D.2.1, D.2.2 and D.2.3

1. Name two decidable real features (with the guaranteeing algorithm) and two undecidable ones (with the shipped evidence method).
2. State HALT precisely, and list three HALT-shaped developer experiences.
3. Reconstruct the diagonal proof: the list, the table, D, and the contradiction.
4. State the counting argument and its conclusion about "most problems".
5. Run the reduction recipe for X = "does this program ever read from stdin?" (Build the wrapper and the equivalence.)
6. Give the three impossible perfect tools and the four evidence-based replacements with their honest promises.

12.7 Answers to self-assessment

1. Tape (unbounded memory of symbols); head (reads/writes one symbol, moves L/R); finite states (with start, accept, reject); transition function (the rule table = the program).
2. Because every behaviour of the machine is a consequence of the table: given state + symbol, the table names the only thing that happens next — write, move, new state. There is no other source of behaviour, exactly as a running program has no behaviour outside its instructions.
3. States q_0 (start, reading the string), q_1 (last symbol was 1), q_2 (last was 0). Rules: q_0,1 → write 1, R, q_1; q_0,0 → write 0, R, q_2; q_1,1 → q_1; q_1,0 → q_2; q_2,1 → q_1; q_2,0 → q_2; q_1,blank → accept; q_2,blank → reject. Trace `01`: q_0 on 0 → q_2; q_2 on 1 → q_1; q_1 on blank → accept (ends in 1 ✓). Trace `10`: q_0 on 1 → q_1; q_1 on 0 → q_2; q_2 on blank → reject (ends in 0 ✗).
4. Equivalence = same set of computable functions; proven by mutual simulation (A simulates B and B simulates A).
5. One tape must sweep across all k simulated tapes per step of the simulated machine; each sweep is proportional to the region of tape used (≤ t cells), so t simulated steps cost about $t \cdot t = t^2$. Quadratic is a step-count (complexity) difference; the languages accepted are identical.
6. The simulator explores all branches — up to 2 choices per step — so t steps can produce 2^t branches; the NTM defines NP (nondeterministic polynomial time).
7. ⟨M⟩ is a fixed encoding of M's transition table (and alphabet, start state) as a string over {0,1}; it is finite because the table is finite. Finiteness is what makes it listable and storable on a tape.
8. 1. Read the current symbol; (2) find the matching rule in ⟨M⟩ (U keeps M's current state on a tape section); (3) perform it: write, move, update M's state, halt if accept/reject; (4) repeat.
9. Memory = tape (instructions ⟨M⟩ and data x together); control unit = U's interpreting rules; registers = the tape section holding M's current state/head position; I/O = reading the initial tape and writing the final one.
10. Decidable: some TM halts on every input, accepting members, rejecting non-members. Recognisable: some TM accepts exactly the members; may loop on non-members. The one-word difference: may.
11. Palindromes: decidable. HALT: recognisable, not decidable. Complement of HALT: not recognisable. Even-1s strings: decidable.
12. A timeout kills the machine after N steps and reports "no answer" — that is a *practical* answer about time, not a *logical* answer about membership; the recogniser's non-members still exist, the machine was just stopped. No finite clock can ever settle membership.
13. Cycle check: instance = graph; algorithm = DFS with a visited set; terminates (each edge visited once); determination = decidable. "Does this program ever print its own source code?" — probe Question 4 (behaviour about programs); undecidable by reduction (wrapper: P′ runs P on x then prints its own source; "P′ prints its own source" ⇔ "P halts on x").

14. Undecidability is proved only by a reduction from a known undecidable problem (or a direct diagonalisation). Failure to find an algorithm is evidence of difficulty, not a logical statement about existence.
15. Decidable: sorting (always terminates). Infeasible: TSP (decidable by trying all tours, but n! grows beyond reach). Undecidable: perfect bug detection (no algorithm can exist).
16. Decidable: email validation (regex/NFA simulation terminates); payment balance check (arithmetic terminates). Undecidable: complete vulnerability detection (shipped: static analysis + signatures); program equivalence (shipped: testing, diffing, restricted equivalence checkers).
17. HALT: given P and x, determine whether P halts on x, with a guaranteed terminating answer for every pair. HALT-shaped experiences: infinite-loop hunts, static analysers' "will this return?" warnings, hung CI tests/jobs.
18. List programs P_1, P_2, …; fill the table (row i, column j) = "does P_i halt on x_j?"; read the diagonal (P_i on x_i); build D that asks the claimed solver about (D, D) and does the opposite; both answers make the solver wrong; no solver exists.
19. Programs are countable (finite strings); languages are uncountable (Cantor's diagonal argument applies to all sets of strings); therefore most problems have no program — the decidable ones are a negligible sliver.
20. Wrapper: from (P, x) build P′ that runs P on x, and *then* performs a read from stdin. "P′ reads from stdin" ⇔ "P halts on x" (P′ reads only after P halts; if P never halts, P′ never reads). A perfect stdin-read detector would decide HALT. Undecidable.
21. Impossible: perfect infinite-loop detector; fully automated program analysis; guaranteed verification of arbitrary programs. Replacements: timeouts/watchdogs/ pattern linters; scoped static analysis + fuzzing/testing (evidence); model checking, bounded verification, verified fixed programs (seL4, CompCert), type systems — each with a named scope.

---

13. Exam-style question bank

(Use these to practise under exam conditions; model answers in Section 13.7. Marks are suggestions — your lecturer sets the real weighting.)

13.1 Multiple choice

1. Which of the following is NOT a component of a Turing machine?
    1. tape (b) head (c) finite set of states (d) a clock with a timeout
2. The "program" of a Turing machine is:
    1. its alphabet (b) its transition function (c) its tape content (d) its number of states
3. A multi-tape Turing machine, compared with a single-tape machine:
    1. can compute more functions (b) can decide more languages (c) is equivalent, but may be faster (d) cannot simulate a single-tape machine

4. Simulating a non-deterministic TM on a deterministic TM costs, in the worst case:
    1. quadratic time (b) linear time (c) exponential time (d) no time — the simulation is free
5. The Universal Turing Machine works because:
    1. it contains every program (b) programs are strings that can be read as data (c) it has infinite states (d) it guesses correctly
6. The von Neumann architecture corresponds to the UTM in that:
    1. the CPU stores the program in registers (b) instructions and data share one memory, fetched and decoded by a fixed control unit (c) programs are wired into the hardware (d) there is one tape per core
7. A language is Turing-decidable if there is a TM that:
    1. accepts the members and may loop on non-members (b) halts on every input, accepting members and rejecting non-members (c) accepts every string (d) runs forever on every input
8. Which of the following languages is Turing-recognisable but NOT decidable?
    1. palindromes (b) strings with even 1s (c) HALT (d) the complement of HALT
9. The four-question determination procedure decides that “does this regex match this text?” is:
    1. undecidable (b) decidable (c) unrecognisable (d) infeasible only
10. The Halting Problem asks:
    1. whether a given program P ever terminates on a given input x (b) how fast a program runs (c) whether a program is efficient (d) how much memory a program uses
11. In the diagonalisation proof, the program D is built to:
    1. simulate all programs in parallel (b) do the opposite of what the claimed solver H predicts about (D, D) (c) halt on every input (d) find the fastest program
12. The counting argument shows that:
    1. there are more programs than problems (b) there are more problems than programs (c) programs and problems are equal in number (d) problems are countable
13. A reduction proving X undecidable must construct, from any (P, x), an X-instance that:
    1. answers “yes” exactly when P halts on x (b) always answers “no” (c) answers faster than P (d) is unrelated to P
14. Which of the following is IMPOSSIBLE?
    1. a linter that flags while-true patterns (b) a static analyser with documented false alarms (c) a tool that guarantees detection of every infinite loop in any program (d) model checking a finite protocol
15. Formal verification (e.g. seL4) succeeds despite the Halting Problem because it:
    1. uses a HALT solver (b) proves a fixed program against a fixed specification — finite work about a finite artefact, not automation over all programs (c) restricts to non-looping programs only (d) runs the program backwards

13.2 Short questions (2–4 marks each)

1. List the four TM components and the real-computer part each models (D.1.1).

2. State why the multi-tape and non-deterministic variants do not change what is computable (D.1.2).
3. Define ⟨M⟩ and state the one fact about programs that makes the UTM possible (D. 1.3).
4. Give the definitions of Turing-decidable and Turing-recognisable in one sentence each (D.1.4).
5. Give the determination (decidable/undecidable) and a one-line reason for: email validation; “does this program halt?” (D.1.5).
6. Name two evidence-based methods real software uses for undecidable requirements (D.1.6).
7. State the Halting Problem and give one developer experience it explains (D.2.1).
8. State the reduction recipe HALT ≤ X in two sentences (D.2.2).
9. Name the three perfect tools that cannot exist (D.2.3).

13.3 Medium questions (5–8 marks each)

1. Write the transition table for a TM that accepts exactly the binary strings with an odd number of 1s, and trace it on `0110` (D.1.1).
2. Explain how a single-tape TM simulates a two-tape TM, and why the slowdown is quadratic rather than exponential (D.1.2).
3. Explain why emulation is “a theorem, not a feature”, connecting the UTM to VMs and interpreters (D.1.3).
4. Explain why waiting can never distinguish “slow” from “forever”, and why this makes the decider/recogniser distinction a *practical* one (D.1.4).
5. Run the four-question procedure on “will this newly submitted code ever run forever in production?” and determine the honest professional deliverable (D.1.5, D.1.6).
6. Explain why a test suite is evidence and not proof, using the counting argument or a reduction (D.1.6).
7. Walk the diagonalisation proof for HALT in five steps, naming each step’s role (D.2.2).
8. Apply the reduction recipe to X = “does this program ever write to a file named log.txt?” and prove X undecidable (D.2.2).
9. Explain why model checking a finite protocol is possible while verifying arbitrary programs is not (D.2.3).

13.4 Long questions (10–15 marks each)

1. **The on-call theorist (D.2.1, D.2.3).** A bank’s batch job hangs at 03:00. (a) Explain why “will it finish?” is the Halting Problem, and why the answer can never be guaranteed for arbitrary jobs. (b) Design the professional engineering response: what policies, tools and promises would you put in place (timeouts, watchdogs, pattern linters, restart rules), and what exactly does each achieve? (c) A vendor offers a “guaranteed hang detector”. Write the procurement questions that the theory tells you to ask, and the two-sentence rejection you would give.

2. **The impossible product (D.2.3, D.1.6).** A startup asks you to build “ProofShield”: a tool that proves any program meets any specification. (a) Give the reduction showing the perfect version is impossible. (b) Redesign the product so it is buildable

and honest: choose a restricted class of programs and properties, name the verification technology (model checking, bounded verification, verified fixed artefacts, type systems), and write the product's honest guarantee. (c) Explain to the CEO, in non-technical language, why "proves everything" is not achievable but "proves this class, always" is a better product.

3. **The diagonal family (D.2.2).** (a) Reconstruct Cantor's proof that the reals are uncountable, and state exactly where the "flip" happens. (b) Reconstruct Turing's proof for HALT, mapping each of Cantor's steps to its Turing version. (c) Use the counting argument to answer: "if I pick a problem at random, how likely is it to be decidable?" (d) Explain why the diagonal argument is constructive, and why that makes the theorem immune to future technology.

4. **The decidable checklist (D.1.4, D.1.5).** A fintech platform's QA team must classify these requirements: (a) "validate this SA ID number"; (b) "decide whether this payment violates any of the 2 000 fraud rules"; (c) "decide whether this new fraud-rule code will ever loop"; (d) "decide whether this log line came from a crashed run". For each: state the instance, the candidate algorithm or reason none can exist, the termination argument, and the determination. For the undecidable one, describe the shipped evidence-based solution.

13.5 Applied/scenario question

1. You are the technical lead for a hospital booking system. The board asks for three "guarantees":
    - **G1:** every booking request returns an answer (approved/declined) within 2 seconds;
    - **G2:** an automated review tool that reads any proposed code change and guarantees it introduces no infinite loop;
    - **G3:** mathematical proof that the scheduling module meets its specification.

    1. Classify each guarantee as possible or impossible, with a one-line theoretical reason (D.1.5, D.2.3). (b) For G2, redesign the promise honestly: which class of code can be checked, which pattern linters and bounded analyses exist, and what the SLA would actually say (D.2.3). (c) For G3, propose the scoped verification strategy: which properties (e.g. "no deadlock in the finite theatre-state model", "terminates within the booking horizon"), which technology (model checking on the finite state space, bounded verification, type checking), and what you would promise the board in the sign-off document (D.1.6, D.2.3). (d) Explain to the board, in plain language, why the honest scope is more valuable than the impossible guarantee — and what the industry practice is (D.2.3, D.1.6).

13.6 Mark allocation suggestion

Question	Type	Marks	Criteria covered
1–15	MCQ	1 each	D.1.1–D.2.3

Question	Type	Marks	Criteria covered
16–24	Short	2–4 each	D.1.1–D.2.3
25–33	Medium	5–8 each	D.1.1–D.2.3
34–37	Long	10–15 each	D.2.1–D.2.3, D.1.4–D.1.6
38	Applied	15	D.1.5, D.1.6, D.2.3

13.7 Model answers (outline form)

1. 1. — a clock is a practical tool, not a TM component.
2. 1. — the transition function.
3. 1. — equivalent, but possibly faster; the equivalence theorem.
4. 1. — up to 2^t branches in the worst case.
5. 1. — programs are strings; ⟨M⟩ is data on the tape.
6. 1. — instructions and data share one memory; fixed control unit fetches and decodes.
7. 1. — halts on every input, accepting members, rejecting non-members.
8. 1. — HALT is recognised by the simulator but not decidable.
9. 1. — regex/NFA simulation always terminates.
10. 1. — whether P ever terminates on x.
11. 1. — does the opposite of H's prediction about (D, D).
12. 1. — more problems than programs; problems are uncountable.
13. 1. — answers "yes" exactly when P halts on x.
14. 1. — a guaranteed all-programs loop detector is a HALT solver.
15. 1. — a finite proof about a fixed artefact; automation over all programs is what fails.
16. Tape = RAM/disk; head = CPU's point of contact (register/PC); states = the program's "where am I / what do I know" memory; transition function = the program/machine code.
17. Each variant is simulable by the plain machine (multi-tape: track encoding, quadratic; NTM: branch simulation, exponential); simulation preserves the accepted languages, so the computable functions are identical.
18. ⟨M⟩ is the finite encoding of M's rules as a string; the one fact: programs are strings, so a machine can read and obey another machine's description.
19. Decidable: a TM halts on every input, accepting members, rejecting non-members. Recognisable: a TM accepts exactly the members; it may loop forever on non-members.
20. Email validation: decidable (regex/NFA simulation terminates). "Does this program halt?": undecidable (diagonalisation/reduction from HALT).
21. Testing and fuzzing (evidence about actual runs); static analysis with documented false alarms; timeouts and watchdogs; restriction of the problem class (model checking, bounded analysis).
22. HALT: given P and x, determine whether P halts on x (guaranteed answer for all pairs). Developer experience: infinite-loop detection, hung CI jobs, "will it return?" warnings — all recogniser/HALT-shaped.

23. Suppose X is decidable. Build a wrapper turning any (P, x) into an X-instance answering “yes” exactly when P halts on x. Then an X-decider decides HALT — contradiction; X is undecidable.
24. Perfect infinite-loop detector; fully automated program analysis; guaranteed verification of arbitrary programs against arbitrary specifications.
25. States: q_0 (even so far), q_1 (odd so far) — the parity checker with accept/reject swapped: q_0,0→q_0,R; q_0,1→q_1,R; q_1,0→q_1,R; q_1,1→q_0,R; q_0,→*reject; q_1,*→accept. Trace `0110`: q_0 on 0→q_0; q_0 on 1→q_1; q_1 on 1→q_0; q_0 on 0→q_0; blank in q_0→reject (two 1s: even → odd-language rejects ✓).
26. Store both tapes as tracks on one tape with head markers; per simulated step, sweep across the used region twice (read all k symbols; write them back with moves). Each sweep is ≤ the used length (≤ t), so t steps cost ≤ t^2 — polynomial; exponential would mean doubling work per step, which simulation does not.
27. Emulation works because a program is data: ⟨M⟩ on the tape lets U behave as M, exactly as a VM/container/interpreter reads the guest program’s bytes and executes them; one fixed host runs any guest — the UTM theorem, not a feature request.
28. A recogniser’s non-members cause unbounded runs; no finite observation of “still running” rules out “will run forever”, so a clock reports elapsed time, never the answer — only a decider (which always halts) can be relied on for “no”.
29. Instance: (code, production inputs) — but “will it ever run forever” asks about arbitrary program behaviour; no candidate algorithm terminates for all inputs (simulation trap); determination: undecidable in general; deliverable: timeout policies, per-job resource limits, watchdog/health endpoints, pattern linters on review, staged rollout with monitoring — evidence-based, scoped.
30. Tests show the bug is absent from tested inputs. Since there are more possible inputs than any finite suite can cover (and more problems than programs), “passed the suite” says nothing logically about untested inputs — it is evidence, valuable but not proof; only restricted-class verification proves.
31. 1. Assume solver H; (2) list all programs P_1, P_2, …; (3) complete the table “P_i halts on x_j?”; (4) build D: ask H about (D, D), do the opposite; (5) both answers contradict D’s behaviour — H fails on (D, D); no solver exists.
32. Wrapper: from (P, x) build P′ that runs P on x, then writes to log.txt. “P′ writes log.txt” ⇔ “P halts on x”. A perfect log-write detector decides HALT — contradiction. Undecidable.
33. Model checking exhaustively explores a *finite* state space — finitely many executions, so the check always terminates (decidable). Arbitrary programs have unbounded state/executions (HALT inside), so no general verifier can terminate correctly on all inputs.
34. 1. “Will the job finish?” on arbitrary job code is HALT — no guaranteed answer; the hang is a recogniser’s silence. (b) Deliver: per-job timeouts with kill-and-report; watchdog heartbeats with restart; linters catching loop patterns at review; capacity/size limits; staged run in a sandbox with monitoring; honest SLAs (“answered within N hours or reported failed”). Each converts the recogniser into a bounded, reportable process. (c) Procurement: “Which program classes does it cover? How does it treat false alarms? What is its false-negative policy? Show the reduction it escapes.” Rejection: any tool that guarantees detection

of every infinite loop would decide HALT, which is impossible — the vendor is selling a pattern detector with a marketing guarantee.

35. 1. Reduction: if V verified any (P, spec) with a yes/no answer, build P′ that runs P on x then must-halt-iff-P-halts construction: P′ halts on all inputs iff P halts on x; V on (P′, “always halts”) would decide HALT. Impossible. (b) Scope: properties = termination-within-budget, type safety, no deadlock; programs = finite-state models (model checking), bounded executions (bounded verification), or a fixed module (dedicated proof); honest guarantee names the class and the bound. (c) “Proving everything” is not harder than “proving this class” — it is a different kind of thing that cannot exist; the scoped proof is machine-checkable, repeatable and worth more in the safety case.

36. 1. List reals as decimals; diagonal = k-th digit of k-th row; flip every digit; new number differs from every row at the diagonal — not listed; no complete list. (b) Same skeleton: list programs; table = halting behaviour; diagonal = P_i on x_i; D flips it; D missing from every complete table. (c) Problems are uncountable ($2^{\aleph_0}$), programs countable ($\aleph_0$); picking uniformly at random, the probability of landing on a decidable problem is 0 — almost every problem has no program. (d) The proof *constructs* D from H; every claimed solver, however clever or future, has its own D — the contradiction is rebuilt on demand, so the theorem cannot be bypassed by new hardware.

37. 1. Instance: ID string; algorithm: Luhn-style checksum + length/format checks; terminates (finite string); decidable. (b) Instance: (transaction, 2 000 rules); algorithm: evaluate rules; terminates (finite rules, finite data); decidable. (c) Instance: (rule code, inputs); probes program behaviour; undecidable (reduction); ship pattern linters, timeouts, fuzz tests. (d) Instance: log line; algorithm: parse and match crash signatures; terminates; decidable (pattern-based; note it is evidence about the signature, not about all crashes).

38. 1. G1: possible — decidable validation pipeline with time budget (engineering, not theory, sets 2 s). G2: impossible — perfect loop detection is HALT; no tool can guarantee it for arbitrary code. G3: possible only scoped — “the scheduling module meets its specification” for arbitrary code is undecidable; scoped verification is possible. (b) G2 honest: CI gate with pattern linters (while-true without exit, unbounded recursion patterns), bounded analyses, timeouts on all tests, fuzz on parser-like inputs; SLA: “flags known loop patterns within the supported language subset; does not claim completeness”. (c) G3: model check the finite theatre/booking state machine for deadlock and invariant violations (decidable — finite model); bounded verification of the scheduling horizon; type checking throughout; sign-off states the scope, the tool, the bound and the residual risks. (d) Plain language: the impossible guarantee is a promise that cannot be kept; the scoped guarantee is a promise that is kept every time — industry ships safety-critical systems exactly this way (model checking, verified kernels, exhaustive tests, monitoring), and the honest scope is what regulators and auditors can rely on.

---

14. Sources and further reading

Standard textbooks

- Sipser, M. (2013). *Introduction to the Theory of Computation* (3rd ed.). Cengage Learning. — Chapter 3 (the Turing machine, variants, the UTM), Chapter 4 (decidable languages, recognisability), Chapter 5 (reducibility, the Halting Problem and its relatives), and Chapter 6 (the recursion theorem, decidability and countability). The definitive treatment of everything in this unit.
- Hopcroft, J. E., Motwani, R. & Ullman, J. D. (2006). *Introduction to Automata Theory, Languages, and Computation* (3rd ed.). Addison-Wesley. — Chapters 8–9 (Turing machines, variants and the UTM) and Chapter 9 (the Halting Problem, undecidability, reductions).
- Turing, A. M. (1936). *On Computable Numbers, with an Application to the Entscheidungsproblem.* Proceedings of the London Mathematical Society, s2-42(1), 230–265. — The original paper: the Turing machine, the universal machine, and the undecidability of the halting problem, all in one article.
- von Neumann, J. (1945). *First Draft of a Report on the EDVAC.* — The stored-program architecture paper, explicitly building on Turing's universal machine.

Foundational and historical context

- Cantor, G. (1891). *Über eine elementare Frage der Mannigfaltigkeitslehre.* — The diagonal argument in its original form (the uncountability of the reals), the ancestor of Turing's proof.
- Davis, M. (ed.) (1965). *The Undecidable: Basic Papers on Undecidable Propositions, Unsolvable Problems and Computable Functions.* Dover. — A collection of the foundational papers (Gödel, Turing, Church, Post) in one volume.
- Petzold, C. (2008). *The Annotated Turing.* Wiley. — A line-by-line guide to Turing's 1936 paper, invaluable for reading the original with modern eyes.

Reference pages consulted for fact-checking (2026)

- Wikipedia: "Turing machine", "Universal Turing machine", "Halting problem", "Decidability (logic)", "Recursively enumerable language", "Recursive language", "Diagonal argument", "Cantor's diagonal argument", "Reduction (recursion theory)", "Stored-program computer", "Von Neumann architecture", "Model checking", "seL4", "CompCert". *(Used to verify the definitions, the equivalence theorems and slowdown factors, the diagonalisation construction, the countability argument, the history — Turing 1936, von Neumann 1945 — and the examples of real verification practice. Cross-checked against Sipser where possible.)*

Module documents (source of truth for outcomes)

- TCR117V_Revised_Learning_Outcomes.md — Outcomes D.1–D.2 and criteria D.1.1–D.1.6, D.2.1–D.2.3.
- Study guide 2026 — module purpose (Section 6) and outline (Section 23).

- Unit A study notes — the stored-program principle and the five features of a computer (Sections 1–2), the Chomsky hierarchy and recursive vs recursively enumerable (Section 5).
- Unit B study notes — the models of computation, Church’s thesis, and the Cantor diagonalisation argument (Sections 5–7), of which this unit is the full formal development.
- Unit C study notes — the classes P and NP, reducibility and NP-completeness (the time-bounded cousin of this unit’s reductions), and heuristics for infeasible problems.

End of Unit D study notes. Next unit: E — Undecidable problems (why perfect tools do not exist).